

1. PROJECT OVERVIEW

1.1 Introduction

A university records office answers one question many times a day: does a student with this registration number exist, and if so, where is the record? With a few hundred students almost any method is fast enough. With a million records, the choice of method decides whether one answer takes a few nanoseconds or more than a millisecond.

This project designs, analyses and tests three search strategies for that question: **linear search** (iterative), **recursive binary search** over a merge-sorted index, and a **hash table with separate chaining**. The theory for each is checked against measurements on 1,000, 10,000, 100,000 and 1,000,000 records generated from my personal seed. Any preparation of the data — sorting it or building an index — is always counted as part of the algorithm's cost.

The main finding is that the hash table answered a search with about 1.3 key comparisons at every size, and, even after paying for its construction, it was the cheapest design once more than about 10–33 searches were made. It is the recommended design, and for the Task 12 workload it is kept, but run as a persistent index with an incremental daily update.

Table 1.1: Project Specifications

Parameter	Description
Project title	Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System
Student name	Jason Joshva M J
Registration number	192511209
Programme	B.E.CSE
Dataset seed	1209 (last four digits of the registration number)
Algorithms	Linear search (iterative); recursive binary search with merge-sort preprocessing; hash table with separate chaining
Implementation	C++17, compiled with g++ 16.2.0 (MSYS2 UCRT64 on Windows) using -O2
GitHub link	
Main objective	Recommend a search design for the record system from both mathematical analysis and measured performance

1.2 Problem Statement

The database holds n student records in the order in which they were entered, so it is not sorted. Each record contains a unique registration number (the search key), a name, a department and

a CGPA. Given a registration number, the system must return the position of the matching record, or report that no such student exists.

Two constraints shape the design. The database may grow from 10^3 to more than 10^6 records, so the method must stay fast as n grows. And any preparation of the data costs time, which has to be added to the search cost before two methods can be compared fairly. In Task 12 the workload changes to 50,000 searches per hour on 1,000,000 records, with new records added once per day.

1.3 Objectives

The objectives follow the twelve assignment tasks:

1. Design three genuinely different search strategies, including at least one iterative and one recursive algorithm (Task 1).
2. Identify the operation that dominates each algorithm's running time and where it occurs (Task 2).
3. Derive each algorithm's operation count and total cost mathematically, preprocessing included (Task 3).
4. Explain which inputs produce the best, average and worst cases, and by how much they differ (Task 4).
5. Compare the growth rates with O , Ω and Θ for 10^3 to 10^6 records (Task 5).
6. Formulate the recurrence of recursive binary search from the code and solve it (Tasks 6 and 7).
7. Implement, test and benchmark all three algorithms on the same personal dataset (Tasks 8 and 9).
8. Compare the measurements with the theory and explain the differences (Task 10).
9. Recommend a design, then re-evaluate it for 50,000 searches per hour (Tasks 11 and 12).

Table 1.2: Tasks and Where Their Evidence Appears

Task	Evidence in this report
Tasks 1–3	Pseudocode, design strategies, basic operations, operation-count derivations (Sections 2–4)
Tasks 4–7	Case analysis with measured inputs, asymptotic comparison, recurrence and its solution (Sections 5–8)
Task 8	Implementation, fairness rules, test plan and correctness output — Figure 1 (Section 9)

Task	Evidence in this report
Task 9	Result tables, terminal output for all four sizes and four graphs — Figures 2–9 (Section 10)
Task 10	Measured growth against predicted growth — Figure 10 (Section 11)
Task 11	Recommendation, trade-offs and break-even analysis — Figure 11 (Section 12)
Task 12	50,000-search benchmark, daily-update benchmark and decision — Figures 12–13 (Section 13)

1.4 Scope of the Project

The project covers the search component of a student record system: data generation, the three algorithms, their analysis, correctness testing, benchmarks at four sizes, graphs, and a redesign for the Task 12 workload. It does not cover a user interface, disk storage or networking; all records are held in main memory.

The personal dataset is generated from seed 1209. My registration number **192511209** is always present in the data (a successful search) and **192511210** is never present (an unsuccessful search), so both cases appear in every test.

1.5 Methodology

1. Generate n unique 9-digit registration numbers from seed 1209 and store the records in random order.
2. Build each algorithm's preprocessing structure: a merge-sorted index for binary search and a hash table for hashing. Linear search uses the records as they are.
3. Send the same 1,000 queries — 500 keys that exist and 500 that do not — to all three algorithms. Count every key comparison and check every answer against an independent answer key.
4. Time each search by repeating the whole query set for at least 200 ms and taking the average; time each preprocessing step the same way.
5. Write the results to CSV files, draw the graphs from those files, and compare them with the formulas derived in Tasks 3–7.
6. For Task 12, repeat the experiment on 1,000,000 records with 50,000 distinct queries and a daily batch of 10,000 new records.

2. TASK 1 — DESIGN THE ALGORITHM

2.1 Problem Definition

Input: an array $DB[0..n-1]$ of student records in arbitrary order, and a search key k (a registration number).

Output: the position i such that $DB[i].regNo = k$, or NOT FOUND if no record has that key.

The three strategies were chosen because they make opposite trade-offs between work done once, before any search, and work done on every search: linear search does nothing in advance and n comparisons per search; binary search sorts once (about $n \log_2 n$ comparisons) and then needs about $\log_2 n$ per search; the hash table spends about n steps building an index and then needs about one comparison per search. They are three different ideas, not iterative and recursive versions of one algorithm.

2.2 Algorithm 1 — Linear Search (Iterative)

2.2.1 Description

Linear search looks at the records one at a time, starting from position 0, and compares each stored registration number with the key. It stops at the first match and returns that position. If it reaches the end of the array without a match, the key is not in the database.

Because the records are in no particular order, looking at one record says nothing about where any other key might be. That is why linear search is the only one of the three methods that works on the raw data with no preparation at all.

2.2.2 Design Strategy

Linear search is a **brute-force** algorithm implemented **iteratively** with a single loop. Its loop keeps one fact true: before position i is examined, the key is known not to be in $DB[0..i-1]$. When i reaches n , the key is known not to be anywhere in the array.

2.2.3 Pseudocode

```
LINEAR-SEARCH( $DB[0..n-1]$ , key)
1  for  $i \leftarrow 0$  to  $n - 1$  do
2      if  $DB[i].regNo = key$  then          > basic operation
3          return  $i$ 
4  return NOT_FOUND
```

2.2.4 Working Principle

In my 1,000-record dataset, registration number 192511209 sits at position 518 (counting from 0). Linear search therefore compares it with 519 records before returning position 518.

Searching for 192511210, which is absent, compares the key with all 1,000 records before reporting NOT FOUND. Both counts appear in the correctness output in Figure 1.

2.2.5 Advantages

- No preprocessing: the first search can start immediately.
- No extra memory beyond the records themselves.
- Works on unsorted data, and a newly added record is searchable at once.
- Short and easy to verify, which makes it a reliable baseline.

2.2.6 Limitations

- The number of comparisons grows in direct proportion to the number of records.
- A search for an absent key always scans the whole database, and "does this student exist?" queries are common in this system.
- Every search repeats the same scan; nothing is learned from earlier searches.
- On my machine, one search on 1,000,000 records took about 1.5 ms (Section 10).

2.2.7 Preprocessing Requirement

None. The preprocessing cost is $P_1(n) = 0$.

2.2.8 Reason for Selection

It is the baseline every other design must beat. It needs nothing in advance, so for a single search on a small table it is hard to improve on, and comparing against it shows exactly how much the other methods gain.

2.3 Algorithm 2 — Recursive Binary Search with Merge-Sort Preprocessing

2.3.1 Description

If the keys are in sorted order, comparing the key with the middle key tells us which half can still contain it. If the key equals the middle key, the search is over. If it is smaller, only the left half can contain it; if it is larger, only the right half. Each recursive call discards half of the remaining range, so the range shrinks to nothing after about $\log_2 n$ calls.

2.3.2 Design Strategy

Binary search is a **decrease-and-conquer** algorithm written **recursively**: each call does a constant amount of work and then solves one subproblem of half the size. (It is often called

divide-and-conquer, but only one of the two halves is ever solved.) The preprocessing step, merge sort, is a true divide-and-conquer algorithm: it solves both halves and combines them.

2.3.3 Pseudocode

```
BUILD-INDEX(DB[0..n-1])
1 for i <- 0 to n - 1 do I[i] <- (DB[i].regNo, i)
2 MERGE-SORT(I, 0, n - 1)           > sort the entries by key
3 return I

MERGE-SORT(I, lo, hi)
1 if lo >= hi then return           > 0 or 1 entry: already sorted
2 mid <- floor((lo + hi) / 2)
3 MERGE-SORT(I, lo, mid)
4 MERGE-SORT(I, mid + 1, hi)
5 MERGE(I, lo, mid, hi)           > at most hi - lo key comparisons

BINARY-SEARCH(I, key, lo, hi)
1 if lo > hi then return NOT_FOUND   > base case: empty range
2 mid <- lo + floor((hi - lo) / 2)
3 if key = I[mid].key then return I[mid].pos > basic operation
4 if key < I[mid].key then return BINARY-SEARCH(I, key, lo, mid - 1)
5 return BINARY-SEARCH(I, key, mid + 1, hi)

first call: BINARY-SEARCH(I, key, 0, n - 1)
```

2.3.4 Working Principle

On the 1,000-record dataset the sorted index has 1,000 entries. The first probe compares the key with entry 499, the next with the middle of whichever half remains, and so on. The search for 192511209 needed 10 probes and the search for 192511210 also needed 10 — close to $\log_2 1,000 \approx 9.97$ (Figure 1).

2.3.5 Preprocessing Requirement

The records are unsorted, so before any search I build an index of (registration number, position) pairs and sort it with merge sort. I sort the 16-byte index entries instead of the 40-byte records so that the records stay where they are and each search touches less memory. Merge sort was chosen because its $\Theta(n \log n)$ bound holds for every input and its comparisons are easy to count. At 1,000,000 records the sort made 18,674,676 comparisons and took 128.152 ms (Section 10).

2.3.6 Advantages

- About $\log_2 n$ comparisons per search, and that bound is guaranteed for every key.
- The sorted index also supports ordered queries, such as listing a range of registration numbers.
- Compact: 16 bytes of index per record.

2.3.7 Limitations

- The data must be sorted first, at a cost of $\Theta(n \log n)$.
- Adding one record to a sorted array means shifting up to n entries, so single inserts are expensive.
- Each probe jumps to a distant part of memory, which is slow once the index no longer fits in the CPU cache.
- The recursion uses one stack frame per probe — at most 20 for a million records, so this is harmless here.

2.3.8 Reason for Selection

It is the standard logarithmic search method, it is naturally recursive (needed for Tasks 6 and 7), and it tests a real question: is a $\Theta(n \log n)$ sort worth paying for in exchange for faster searches?

2.4 Algorithm 3 — Hash Table with Separate Chaining

2.4.1 Description

Instead of searching for the key, a hash table computes where the key must be. A hash function $h(k) = k \bmod m$ sends every registration number to one of m buckets. Each bucket holds a short chain (a linked list) of the keys that were sent to it. A search computes $h(\text{key})$ and then scans only that one chain.

2.4.2 Design Strategy

This is a **transform-and-conquer** (space-for-time) strategy: the data is changed once into a representation in which searching is cheap. The lookup itself is **iterative** — a loop along one chain. The number of buckets m is the smallest prime $\geq n / 0.75$, so the load factor $\alpha = n/m$ stays near 0.75. The chains are stored in arrays (`head[]`, `next[]`, `keys[]`, `pos[]`) rather than as separate list nodes, which keeps memory compact.

2.4.3 Pseudocode

```
BUILD-HASH(DB[0..n-1])
1  m <- NEXT-PRIME(ceil(n / 0.75))
2  for b <- 0 to m - 1 do head[b] <- -1          > every bucket starts empty
3  for e <- 0 to n - 1 do
4      keys[e] <- DB[e].regNo; pos[e] <- e
5      b <- keys[e] mod m
6      next[e] <- head[b]; head[b] <- e        > insert at the front of the chain

HASH-SEARCH(key)
1  b <- key mod m
2  e <- head[b]
3  while e != -1 do
4      if keys[e] = key then return pos[e]      > basic operation
5      e <- next[e]
6  return NOT_FOUND
```

2.4.4 Working Principle

For $n = 1,000$ the table has $m = 1,361$ buckets, so $\alpha = 1,000 / 1,361 = 0.735$. My registration number 192511209 lands in bucket $192511209 \bmod 1,361 = 481$, where it is the second key in the chain, so the search needs 2 comparisons. Reg no + 1, 192511210, lands in bucket 482, which happens to be empty, so it is rejected with 0 comparisons (Figure 1).

2.4.5 Why m Is a Prime Number

Real registration numbers share structure: the same year and department digits followed by serial numbers. With a prime modulus, consecutive serial numbers fall into consecutive buckets and never collide with each other, and shared digit patterns do not pile keys into a few buckets.

2.4.6 Preprocessing Requirement

Building the table sets m bucket heads to empty and inserts n keys at the front of their chains, each in constant time. At 1,000,000 records this took 13.872 ms and uses about 21.3 MB (4 bytes per bucket plus 16 bytes per entry).

2.4.7 Advantages

- About $1 + \alpha/2 \approx 1.4$ comparisons per successful search, independent of n .
- The build is a single $\Theta(n)$ pass — cheaper than sorting.
- A new record is inserted in constant amortised time, with no shifting.

2.4.8 Limitations

- The guaranteed worst case is $\Theta(n)$: if every key fell into the same bucket, one search would scan them all.
- About 33% more memory than the sorted index.
- No ordering, so range queries are not possible.

- The table must be enlarged (rehashed) when α grows too high.

2.4.9 Reason for Selection

It is the strategy assigned to my roll-number group (Group A), and it tests the opposite trade-off to binary search: spend $\Theta(n)$ time and extra memory once so that each search costs a constant expected amount.

2.5 Comparison of the Three Algorithms

Table 2.1: Comparison of the Three Designs

Feature	Linear search	Binary search	Hash table
Implementation style	Iterative	Recursive	Iterative lookup
Design strategy	Brute force	Decrease-and-conquer (+ merge sort)	Transform-and-conquer (space for time)
Data order needed	None	Sorted index	None
Preprocessing	None	Build index and merge sort it: $\Theta(n \log n)$	Insert all keys into $m \approx n/0.75$ buckets: $\Theta(n)$
Extra memory at 10^6 records	None	16 MB (+16 MB while sorting)	21.3 MB
Comparisons per search	$(n+1)/2$ found, n absent	$\approx \log_2 n$	$\approx 1 + \alpha/2$ found, α absent
Guaranteed worst case	$\Theta(n)$	$\Theta(\log n)$	$\Theta(n)$ (all keys in one bucket)
Ordered (range) queries	Slow	Supported	Not supported
Adding one record	$\Theta(1)$ append	$\Theta(n)$ shift	$\Theta(1)$ amortised insert

No design wins on every row: linear search wins on setup, binary search on guaranteed worst case and ordering, and the hash table on search cost. Which matters most depends on how many searches are made per update — the question Tasks 11 and 12 answer with measurements.

3. TASK 2 — IDENTIFY THE BASIC OPERATION

3.1 Introduction

In all three algorithms the basic operation is the **comparison of the search key with a stored registration number**. What differs is how many times that comparison runs as n grows: up to n times, about $\log_2 n$ times, or a constant number of times.

A key comparison is the right operation to count for three reasons. It sits in the innermost loop (linear search, hash lookup) or runs once per recursive call (binary search). Every other statement — the loop counter, computing mid, the recursive call, following a next pointer — runs at most a fixed number of times per comparison, so the running time is roughly $c \times (\text{number of comparisons}) + \text{a small constant}$. And the count does not depend on the computer, so the program can count it exactly: each search function adds 1 to a counter on the same line as the comparison.

3.2 Basic Operation in Linear Search

Operation: `db[i].reg_no == key`, inside the for loop of `linear_search` (`src/algo1_linear.cpp`), executed once for every record examined.

```
int linear_search(const std::vector<Record>& db, long long key, long long& comparisons) {
    int n = (int)db.size();
    for (int i = 0; i < n; i++) {
        comparisons++;
        if (db[i].reg_no == key) { // <-- BASIC OPERATION
            return i;
        }
    }
    return -1;
}
```

Listing 1: The basic operation of linear search (highlighted line), from `src/algo1_linear.cpp`.

How the count grows: between 1 (key in the first record) and n (key last or absent). Ten times as many records means ten times as many comparisons.

3.3 Basic Operation in Recursive Binary Search

Operation: `key == index[mid].key`, followed if needed by `key < index[mid].key` on the same pair, in `binary_search_recursive` (`src/algo2_binary.cpp`), executed once per recursive call.

```

int binary_search_recursive(const std::vector<IndexEntry>& index, long long key,
                           int lo, int hi, long long& comparisons) {
    if (lo > hi) return -1; // base case: empty range
    int mid = lo + (hi - lo) / 2;
    comparisons++; // one three-way comparison
    if (key == index[mid].key) // <-- BASIC OPERATION
        return index[mid].pos;
    if (key < index[mid].key)
        return binary_search_recursive(index, key, lo, mid - 1, comparisons);
    return binary_search_recursive(index, key, mid + 1, hi, comparisons);
}

```

Listing 2: The basic operation of recursive binary search (highlighted line), from `src/algo2_binary.cpp`.

Counting convention: the equality test and the less-than test compare the same two values and compile to one comparison instruction, so I count one three-way comparison (one *probe*) per call. Counting it as two would double the constant but not change how the count grows.

How the count grows: between 1 (key at the first midpoint) and $\lceil \log_2 n \rceil + 1$. Ten times as many records adds only about 3.3 comparisons.

3.4 Basic Operation in Hash Table Search

Operation: `h.keys[e] == key`, inside the chain loop of `hash_search` (`src/algo3_hash.cpp`), executed once for every chain entry scanned.

```

int hash_search(const HashTable& h, long long key, long long& comparisons) {
    long long b = key % h.m; // 1. jump to the bucket
    for (int e = h.head[b]; e != -1; e = h.next[e]) { // 2. walk its chain
        comparisons++;
        if (h.keys[e] == key) // <-- BASIC OPERATION
            return h.pos[e];
    }
    return -1;
}

```

Listing 3: The basic operation of the hash-table lookup (highlighted line), simplified from `src/algo3_hash.cpp`.

How the count grows: between 0 (empty bucket) and the length of the chain; on average about $1 + \alpha/2$ for a key that exists and α for one that does not. Because m grows with n , α stays near 0.75 and the count does not grow at all. Each search also computes one remainder, `key % m`; that happens exactly once whatever n is, so it adds a constant rather than a growing term.

3.5 Basic Operations of the Preprocessing Steps

Preprocessing is analysed separately. For binary search the basic operation is the key comparison inside MERGE (`a[i].key <= a[j].key`), which runs about $n \log_2 n$ times. For the hash table it is one insertion — one remainder plus two array writes — which runs exactly n times.

Table 3.1: Basic Operation Summary

Algorithm	Basic operation	Where it runs	Executions per search	Growth with n
Linear search	db[i].reg_no == key	linear_search, once per record examined	1 to n	Linear
Binary search	key == index[mid].key	binary_search_recursive, once per call	1 to $\lfloor \log_2 n \rfloor + 1$	Logarithmic
Hash table	h.keys[e] == key	hash_search, once per chain entry	0 to chain length; $\approx 1 + \alpha/2$ found	Constant (α fixed)
Merge sort (A2 preprocessing)	a[i].key <= a[j].key	merge step of merge_sort	$\approx n \log_2 n$ per build	$n \log n$
Hash build (A3 preprocessing)	one insertion	hash_build loop	n per build	Linear

4. TASK 3 — DERIVE THE TIME-COMPLEXITY FUNCTION MATHEMATICALLY

Notation. n = number of records (the input size); $C(n)$ = key comparisons for one search; $P(n)$ = preprocessing cost; k = number of searches made after preprocessing; p = probability that the searched key is present.

4.1 Linear Search

If the key is at position i (counting from 1), the loop makes exactly i comparisons. If every position is equally likely, each has probability $1/n$, so the average for a key that is present is:

$$C_{\text{found}}(n) = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

A key that is absent is compared with every record, so $C_{\text{absent}}(n) = n$. If the key is present with probability p :

$$C_{\text{avg}}(n) = p \cdot \frac{n+1}{2} + (1-p) \cdot n, \quad \text{for } p = \frac{1}{2}: \quad C_{\text{avg}}(n) \approx 0.75n$$

There is no preprocessing ($P_1(n) = 0$), so the running time of one search is $T_1(n) = c_1 \cdot C(n) + c_0 \in \Theta(n)$, and k searches cost:

$$\text{Total}_1(n, k) = k \cdot c_1 \cdot C_{\text{avg}}(n) \in \Theta(kn)$$

4.2 Recursive Binary Search with Merge-Sort Preprocessing

Preprocessing

Copying the n keys into the index costs $\Theta(n)$. Merge sort splits the index in half, sorts both halves and merges them with at most $n - 1$ comparisons. For $n = 2^i$, in the worst case:

$$M(n) = 2M(n/2) + (n - 1), \quad M(1) = 0$$

Expanding the recurrence i times gives $M(n) = 2^i \cdot M(n/2^i) + i \cdot n - (2^i - 1)$. Stopping at $i = \log_2 n$, where $M(1) = 0$:

$$M(n) = n \log_2 n - n + 1 \in \Theta(n \log n)$$

On random keys a merge often empties one half early, so the average is slightly lower, about $n \log_2 n - 1.25n$. At $n = 10^6$ that predicts 18,681,569 comparisons; my program counted 18,674,676, a difference of 0.04%. So $P_2(n) = \Theta(n) + \Theta(n \log n) = \Theta(n \log n)$.

Search

Each probe halves the remaining range — $n, \lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \dots$ — until it is empty, so the worst case is $\lfloor \log_2 n \rfloor + 1$ probes (proved in Tasks 6 and 7). For the average successful search, view the probes as a balanced decision tree: 1 key is found after 1 probe, 2 keys after 2 probes, 4 keys after 3 probes, and 2^{j-1} keys after j probes. For $n = 2^h - 1$:

$$C_{\text{found}}(n) = \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{(h-1)2^h + 1}{n} \approx h - 1 = \log_2(n+1) - 1$$

An unsuccessful search always runs off the bottom of the tree, after about $\log_2(n+1)$ probes. So one search is $T_2(n) \in \Theta(\log n)$, and preprocessing plus k searches costs:

$$\text{Total}_2(n, k) = c_s n \log_2 n + k c_b \log_2 n \in \Theta(n \log n + k \log n)$$

4.3 Hash Table with Separate Chaining

Preprocessing

Finding the prime $m \approx n/0.75$ tests a handful of numbers with at most $\sqrt{m}$ trial divisions each, which is negligible. Setting the m bucket heads to empty costs $\Theta(m)$ and the n front-insertions cost $\Theta(1)$ each, so $P_3(n) = \Theta(n + m) = \Theta(n)$, because $m \approx 1.33n$.

Search

Assume simple uniform hashing: every key is equally likely to land in any of the m buckets. The expected chain length is then $\alpha = n/m$. An absent key is compared with its whole chain, so $C_{\text{absent}} = \alpha$. For a present key, recall that keys are inserted at the front: searching for the i -th inserted key costs 1 plus the number of keys inserted after it into the same bucket. Each of the $n - i$ later keys shares its bucket with probability $1/m$, so:

$$C_{\text{found}} = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

With $\alpha = 0.75$ this is about 1.375 comparisons for a present key and 0.75 for an absent one, plus one remainder calculation. Because m grows with n , α — and therefore the cost — does not depend on n : $T_3(n) \in \Theta(1 + \alpha) = \Theta(1)$ expected. Preprocessing plus k searches costs:

$$\text{Total}_3(n, k) = c_h n + k c (1 + \alpha/2) \in \Theta(n + k)$$

4.4 Summary and Break-Even Point

Table 4.1: Derived Cost Functions

Algorithm	Preprocessing $P(n)$	Comparisons per search $C(n)$	Total for k searches
Linear search	0	$(n+1)/2$ found; n absent	$\Theta(kn)$
Binary search	$\approx n \log_2 n - 1.25n$ comparisons: $\Theta(n \log n)$	$\approx \log_2(n+1) - 1$ found; $\approx \log_2(n+1)$ absent	$\Theta(n \log n + k \log n)$
Hash table	n insertions + m initialisations: $\Theta(n)$	$1 + \alpha/2$ found; α absent	$\Theta(n + k)$

Preprocessing pays for itself once it is cheaper than the linear searches it replaces. For the hash table the condition is $c_h \cdot n < k \cdot 0.75 \cdot c_1 \cdot n$, that is, $k > c_h / (0.75 \cdot c_1)$: a **constant**, because both the build and one linear search are $\Theta(n)$. For the sorted index it is $k > c_s \cdot \log_2 n / (0.75 \cdot c_1)$, which grows slowly with n . Section 10 measures both break-even points.

5. TASK 4 — BEST, AVERAGE AND WORST-CASE BEHAVIOUR

5.1 Introduction

Each algorithm is sensitive to a different property of the input: linear search to where the key sits in the array, binary search to where it sits in sorted order, and the hash table to how the keys are spread over the buckets. That is why their best and worst cases are triggered by different inputs. Every case below is also shown with a real key from my 1,000,000-record dataset in Table 5.2.

5.2 Linear Search — Cost Equals the Key's Position

Best case — the key is the first record, DB[0]: 1 comparison. The scan stops at once. With records in random order this happens with probability only $1/n$.

$$C_{\text{best}}(n) = 1 \quad \Rightarrow \quad T_{\text{best}}(n) = \Theta(1)$$

Worst case — the key is the last record, or not in the database: n comparisons. An unsorted array gives no sign that the key is missing until every record has been checked. Absent keys are common here ("does this student exist?"), so the worst case is not rare.

$$C_{\text{worst}}(n) = n \quad \Rightarrow \quad T_{\text{worst}}(n) = \Theta(n)$$

Average case — a present key equally likely to be at any position: $(n+1)/2$ comparisons; if the key is present with probability p , the average is $p(n+1)/2 + (1-p)n$ (Section 4.1). Doubling n doubles every case except the best.

$$C_{\text{avg}}(n) = \frac{n+1}{2} \quad \Rightarrow \quad T_{\text{avg}}(n) = \Theta(n)$$

5.3 Recursive Binary Search — Cost Equals the Key's Depth in the Decision Tree

Best case — the key equals the first midpoint, $I[(n-1)/2]$: 1 comparison.

Worst case — a key on the longest path, for example an absent key larger than every key: $\lceil \log_2 n \rceil + 1$ comparisons. Each probe halves the range, and an unsuccessful search ends only when the range is empty.

Average case — about $\log_2(n+1) - 1$ comparisons for a present key and about $\log_2(n+1)$ for an absent one. The average is only about one probe below the worst case, because half of all keys sit on the deepest level of the decision tree.

Preprocessing has cases too, but is always $\Theta(n \log n)$. A merge of two halves costs between half and all of its length in comparisons, so merge sort needs about $\frac{1}{2} \cdot n \log_2 n$ comparisons on

keys that are already sorted and up to $n \log_2 n - n + 1$ in the worst case. It never drops below order $n \log n$, so the full sorting cost is paid whatever the input order.

5.4 Hash Table — Cost Equals the Length of the Key's Chain

Best case — the key is at the head of its chain: 1 comparison; or an absent key whose bucket is empty: 0 comparisons. With $\alpha = 0.75$ a bucket is empty with probability about $e^{-0.75} \approx 0.47$, so nearly half of all absent keys are rejected without a single key comparison.

Average case — $1 + \alpha/2 \approx 1.375$ comparisons for a present key and $\alpha \approx 0.75$ for an absent one, the same at every n because α is held fixed (Section 4.3).

Worst case in practice — the last key of the longest chain. With n keys in about $1.33n$ buckets the longest chain grows extremely slowly; my runs found longest chains of 6, 6, 8 and 7 entries at $n = 10^3, 10^4, 10^5$ and 10^6 .

Worst case in theory — every key in one bucket: n comparisons, $\Theta(n)$. This needs keys that all leave the same remainder modulo m , such as keys exactly m apart. A prime m unrelated to the structure of registration numbers makes this practically impossible for real data, but it remains the formal worst case.

Table 5.1: Best-, Average- and Worst-Case Complexity of One Search

Algorithm	Best case	Average case	Worst case
Linear search	$\Theta(1)$: key = DB[0]	$\Theta(n)$: $(n+1)/2$ found	$\Theta(n)$: last or absent key
Binary search	$\Theta(1)$: key = first midpoint	$\Theta(\log n)$: $\approx \log_2(n+1) - 1$	$\Theta(\log n)$: $\lfloor \log_2 n \rfloor + 1$
Hash table	$\Theta(1)$: head of chain (0 if bucket empty)	$\Theta(1)$ expected: $1 + \alpha/2$	$\Theta(n)$: all keys in one bucket

Table 5.2: Measured Case Counts for $n = 1,000,000$ (from Figure 5)

Algorithm	Case and input that triggers it	Search key	Theory	Measured comparisons
Linear	Best: key is the first record	240975095	1	1
Linear	Worst: key is the last record	892733869	1,000,000	1,000,000
Linear	Worst: my reg no + 1 (absent)	192511210	1,000,000	1,000,000
Linear	My reg no (found)	192511209	its position = 339,457	339,457
Linear	Average of 500 present keys	—	500,000.5	507,564.13
Binary	Best: key at the middle of the index	550171466	1	1
Binary	Worst: key above the largest key	999999525	20	20

Algorithm	Case and input that triggers it	Search key	Theory	Measured comparisons
Binary	My reg no + 1 (absent)	192511210	≈ 19.93	20
Binary	My reg no (found)	192511209	≤ 20	20
Binary	Average of 500 present keys	—	18.93	18.99
Hash	Best: key at the head of a chain	494675449	1	1
Hash	Best absent: key whose bucket is empty	100001775	0	0
Hash	Worst observed: end of the longest chain	158843242	longest chain = 7	7
Hash	My reg no + 1 (absent)	192511210	its chain length = 1	1
Hash	My reg no (found)	192511209	its place in its chain	2
Hash	Average of 500 present keys	—	1.375	1.31

Every best and worst case matched its prediction exactly. The averages differ slightly from theory because they are taken over 500 random keys; Section 11 shows that the differences are within normal sampling variation. All four sizes are in results/cases.csv.

6. TASK 5 — PERFORM ASYMPTOTIC ANALYSIS

6.1 Introduction

When the database grows 1,000-fold, from 10^3 to 10^6 records, a linear search does 1,000 times more work, a binary search makes 10 more comparisons, and a hash lookup does the same work. In asymptotic terms the search costs are ordered $\Theta(1) < \Theta(\log n) < \Theta(n)$, while the preprocessing costs run the other way: none, $\Theta(n \log n)$ and $\Theta(n)$.

6.2 Upper, Lower and Tight Bounds

O (upper bound) holds for every input of size n ; **Ω (lower bound)** holds for every input; **Θ (tight bound)** applies when both match. No single Θ covers every input of linear or binary search, because their best case is $\Theta(1)$ and their worst case is not, so Θ is stated per case.

Table 6.1: Asymptotic Bounds

Algorithm	Best	Average	Worst	Overall bound	Preprocessing
Linear search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$O(n), \Omega(1)$	None
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n), \Omega(1)$	$\Theta(n \log n)$ always
Hash table	$\Theta(1)$	$\Theta(1)$ expected	$\Theta(n)$ (one bucket)	$O(n)$ worst, $\Theta(1)$ expected	$\Theta(n)$

6.3 Showing the Bounds from the Task 3 Counts

- Linear search, present key: $\frac{1}{2} \cdot n \leq (n+1)/2 \leq n$ for all $n \geq 1$, so $C(n) \in \Theta(n)$ with $c_1 = \frac{1}{2}$, $c_2 = 1$, $n_0 = 1$.
- Binary search, worst case: $\log_2 n \leq \lfloor \log_2 n \rfloor + 1 \leq 2 \cdot \log_2 n$ for all $n \geq 2$, so $C(n) \in \Theta(\log n)$ with $c_1 = 1$, $c_2 = 2$, $n_0 = 2$.
- Hash table, present key: $1 \leq 1 + \alpha/2 \leq 1.5$ for any $\alpha \leq 1$, so $C(n) \in \Theta(1)$.
- Ordering: $\log_2 n / n \rightarrow 0$ as $n \rightarrow \infty$, so $\log n$ grows strictly more slowly than n ; $(n \log n)/n = \log n \rightarrow \infty$, so sorting grows strictly faster than one linear scan.

6.4 Growth from 10^3 to 10^6 Records

Table 6.2: Theoretical Comparison Counts as the Database Grows

n	$\log_2 n$	Linear found (n+1)/2	Linear worst n	Binary found $\log_2(n+1)-1$	Binary worst $\lceil \log_2 n \rceil + 1$	Hash found $1+\alpha/2$	Merge sort $\approx n \log_2 n$
1,000	9.97	500.5	1,000	8.97	10	1.375	9,966
10,000	13.29	5,000.5	10,000	12.29	14	1.375	132,877
100,000	16.61	50,000.5	100,000	15.61	17	1.375	1,660,964
1,000,000	19.93	500,000.5	1,000,000	18.93	20	1.375	19,931,569

6.5 What the Growth Means for the Record System

- **Linear search stops being practical as the university grows.** At 10^6 records an average successful search needs half a million comparisons — about 26,000 times more than binary search and 360,000 times more than the hash table. Every tenfold growth in students makes every search ten times slower.
- **Binary search barely notices growth.** Going from 1,000 to 1,000,000 records adds only 10 comparisons, and 10^9 records would need just 30. Its cost sits in the $\Theta(n \log n)$ sort: about 18.7 million comparisons at 10^6 records, roughly the work of 37 average linear searches.
- **The hash table's search cost does not grow at all**, as long as m is enlarged with n so that α stays fixed. Its build is n insertions — at 10^6 records about the work of two linear searches.
- **Asymptotic order is not the whole story.** Θ hides constant factors and memory behaviour; Section 11 checks how closely real times follow these orders.

7. TASK 6 — FORMULATE THE RECURRENCE RELATION

7.1 Introduction

Reading the code of `binary_search_recursive` gives the worst-case recurrence $T(n) = T(\lfloor n/2 \rfloor) + 1$ with $T(0) = 0$, where $T(n)$ is the number of key comparisons on a range of n entries. The derivation below follows the code line by line.

```
int binary_search_recursive(index, key, lo, hi, c) {    // c = comparison counter
    if (lo > hi) return -1;                          // (1) base case
    int mid = lo + (hi - lo) / 2;                    // (2) constant work
    c++;
    if (key == index[mid].key) return index[mid].pos; // (3) one comparison
    if (key < index[mid].key)
        return binary_search_recursive(index, key, lo, mid - 1, c); // (4a) left part
    return binary_search_recursive(index, key, mid + 1, hi, c);    // (4b) right part
}
```

7.2 Components of the Recurrence

Table 7.1: Components of the Binary Search Recurrence

Component	From the code	Value
Input size of the original problem	number of entries in the range [lo, hi]	$n = hi - lo + 1$ (first call: the whole index)
Size of the recursive subproblem	left part [lo, mid-1] or right part [mid+1, hi]	left: $\lfloor (n-1)/2 \rfloor$; right: $\lfloor (n-1)/2 \rfloor = \lfloor n/2 \rfloor$
Number of recursive calls	only one of lines (4a) and (4b) runs	1 — only one half can contain the key
Work outside the recursive call	lines (2) and (3): compute mid, one three-way comparison	1 comparison, constant time c
Base case	line (1): empty range, no comparison	$T(0) = 0$

Checking the subproblem sizes. With $n = 4$ ($lo = 0$, $hi = 3$): $mid = 1$, the left part has 1 entry and the right part has $2 = \lfloor 4/2 \rfloor$. With $n = 5$ ($lo = 0$, $hi = 4$): $mid = 2$, and both parts have $2 = \lfloor 5/2 \rfloor$ entries. So the larger part always has $\lfloor n/2 \rfloor$ entries.

7.3 Putting the Components Together

The worst case is when the key is never found at mid and the search continues into the larger part every time: one comparison, plus the cost of one recursive call on $\lfloor n/2 \rfloor$ entries.

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \quad T(0) = 0 \quad (\text{so } T(1) = 1)$$

In units of time the same recurrence reads $T(n) = T(\lfloor n/2 \rfloor) + c$. If the key is found at the first midpoint, the recursion stops after one comparison, so the best case is $T(n) = 1$. For n a power

of two the worst case simplifies to $T(n) = T(n/2) + 1$ with $T(1) = 1$, which is the form used in Section 8.

The recurrence has one recursive term, not two, because the algorithm discards one half without looking at it. It describes the search only; the merge-sort preprocessing has its own recurrence, $M(n) = 2M(n/2) + (n - 1)$, from Section 4.2.

8. TASK 7 — SOLVE THE RECURRENCE RELATION

The recurrence solves to exactly $T(n) = \lfloor \log_2 n \rfloor + 1$ comparisons, which is $\Theta(\log n)$. I solve it three ways: iteration finds the formula, the Master Theorem confirms its order, and substitution proves the bounds for every n , floors included.

8.1 Method 1 — Iteration (Unrolling)

Iteration suits this recurrence because it has only one recursive term, so unrolling produces a single chain rather than a tree. For $n = 2^k$:

$$T(n) = T(n/2) + 1 = T(n/4) + 2 = T(n/8) + 3 = \dots = T(n/2^i) + i$$

The chain reaches the base case $T(1) = 1$ when $n/2^i = 1$, that is, when $i = \log_2 n$:

$$T(n) = T(1) + \log_2 n = \log_2 n + 1$$

For any n , not only powers of two: the range sizes visited are n , $\lfloor n/2 \rfloor$, $\lfloor n/4 \rfloor$, ..., and $\lfloor n/2^i \rfloor \geq 1$ exactly when $2^i \leq n$, that is, $i \leq \lfloor \log_2 n \rfloor$. So there are $\lfloor \log_2 n \rfloor + 1$ non-empty calls, each making one comparison:

$$T(n) = \lfloor \log_2 n \rfloor + 1$$

For $n = 10^6$ this gives $\lfloor 19.93 \rfloor + 1 = 20$, exactly the worst case measured in Table 5.2.

8.2 Method 2 — Master Theorem

The Master Theorem applies to recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$ with constants $a \geq 1$, $b > 1$ and $f(n)$ asymptotically positive. Binary search fits it: one subproblem ($a = 1$) of half the size ($b = 2$) with constant work outside the call ($f(n) = \Theta(1)$). The floor in $\lfloor n/2 \rfloor$ does not affect the result.

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1, \quad f(n) = \Theta(1) = \Theta(n^{\log_b a})$$

Since $f(n)$ grows at the same rate as $n^{\log_2 1} = 1$, this is Case 2 of the theorem:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(\log n)$$

The same theorem confirms the preprocessing cost: merge sort has $M(n) = 2M(n/2) + \Theta(n)$, so $a = 2$, $b = 2$, $n^{\log_2 2} = n = \Theta(f(n))$ — again Case 2, giving $\Theta(n \log n)$.

8.3 Method 3 — Substitution (Proof for Every $n \geq 1$)

Upper bound: $T(n) \leq \log_2 n + 1$. Base: $T(1) = 1 \leq 0 + 1$. Step: assume the bound holds for every size below n ($n \geq 2$). Then:

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \leq \log_2 \lfloor n/2 \rfloor + 2 \leq \log_2 (n/2) + 2 = \log_2 n + 1$$

Lower bound: $T(n) \geq \log_2(n + 1)$. Base: $T(1) = 1 = \log_2 2$. Step: since $2\lfloor n/2 \rfloor + 2 \geq n + 1$ for every n :

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \geq \log_2 (\lfloor n/2 \rfloor + 1) + 1 = \log_2 (2\lfloor n/2 \rfloor + 2) \geq \log_2 (n + 1)$$

Therefore $\log_2(n + 1) \leq T(n) \leq \log_2 n + 1$, and $T(n) \in \Theta(\log n)$.

8.4 What the Result Says About the Algorithm

- **Doubling the database adds exactly one comparison** in the worst case. Going from 1,000 to 1,000,000 records takes it from 10 to 20, as measured.
- **The recursion is shallow.** The call depth equals $T(n)$: at most 20 stack frames for a million records and 30 for a billion, so the recursion uses $O(\log n)$ stack space and cannot overflow here.
- **Each call does constant work and ends in a single tail call**, so with -O2 the compiler can turn the recursion into a loop. The recursive form describes the algorithm; it does not add measurable overhead.
- **The search is cheap; the sort is not.** $\Theta(\log n)$ per search sits on top of a $\Theta(n \log n)$ sort, so binary search pays off only when the sorted index is reused for many searches.

9. TASK 8 — IMPLEMENT AND TEST THE ALGORITHMS

9.1 Implementation Overview

All three algorithms are implemented in C++17 exactly as in the Task 1 pseudocode, each in its own file. All 19 test checks passed, including random cross-checks against an independent answer key at all four dataset sizes (Figure 1).

Table 9.1: Source Files and What They Contain

File	Contents	Matches
src/algo1_linear.cpp	linear_search	LINEAR-SEARCH
src/algo2_binary.cpp	build_sorted_index, merge_sort, binary_search_recursive, merge_batch_into_index	BUILD-INDEX, MERGE-SORT, BINARY-SEARCH
src/algo3_hash.cpp	next_prime, hash_build, hash_search, hash_insert	BUILD-HASH, HASH- SEARCH
src/dataset_generator.cpp	Seeded generator of unique student records	—
src/benchmark.cpp	The test, bench and task12 commands: counters, timing, CSV output	—
src/config.h	My registration number and name	—
src/plot_results.py	Graphs drawn from my own CSV files	—

Every search function takes a comparison counter and adds 1 on the line of its basic operation, so the counts reported in Sections 5 and 10 come from the very lines analysed in Sections 3 and 4.

9.2 Dataset Configuration

Table 9.2: Dataset and Test Configuration

Parameter	Value
Student name	Jason Joshva M J
Registration number	192511209
Dataset seed	1209 (last four digits of the registration number)
Key format	9-digit registration numbers, 100000000 to 999999999, all unique
Successful search key	192511209 (always inserted into the dataset)
Unsuccessful search key	192511210 (never inserted)
Record layout	40 bytes: reg no, name, department, CGPA — stored in random (shuffled) order
Dataset sizes	1,000; 10,000; 100,000; 1,000,000
Queries per size	1,000: 500 present keys + 500 absent keys, the same for every algorithm

Parameter	Value
Language and build	C++17, g++ 16.2.0, -O2, MSYS2 UCRT64 on Windows
Test machine	CPU model, RAM, Windows version

9.3 Keeping the Comparison Fair

- **Same data.** One generated array of records is used by all three algorithms. Binary search and the hash table build their structures from that same array, and every algorithm returns a position in it.
- **Same queries.** At each size all algorithms answer the same 1,000 queries, mixed in random order.
- **Same conditions.** One machine, one compiler setting and one program run per size. Each search time is the average over repeated passes of the query set lasting at least 200 ms, so even very fast searches are timed accurately.
- **Reproducible.** The generator uses mt19937_64 with its own mapping and shuffle, so the same registration number produces the same dataset and the same comparison counts on any compiler or operating system. Only the times depend on the machine.

9.4 Correctness Testing

Running `./daa test` checks the ten fixed cases below on the 1,000-record dataset, two edge cases, and a random cross-check at all four sizes. A case passes only if all three algorithms give the expected answer, return the same position, and that position really holds the searched registration number. The numbers in brackets are key comparisons.

Table 9.3: Fixed Test Cases on the n = 1,000 Dataset (actual results)

#	Test case	Key	Expected	Linear	Binary	Hash	Result
1	My reg no	192511209	FOUND	FOUND (519)	FOUND (10)	FOUND (2)	PASS
2	My reg no + 1	192511210	NOT FOUND	NOT FOUND (1000)	NOT FOUND (10)	NOT FOUND (0)	PASS
3	First record (DB order)	762163974	FOUND	FOUND (1)	FOUND (8)	FOUND (2)	PASS
4	Last record (DB order)	663093799	FOUND	FOUND (1000)	FOUND (10)	FOUND (1)	PASS
5	Smallest reg no	100040148	FOUND	FOUND (234)	FOUND (9)	FOUND (1)	PASS
6	Largest reg no	999773242	FOUND	FOUND (384)	FOUND (10)	FOUND (1)	PASS
7	Below smallest (min – 1)	100040147	NOT FOUND	NOT FOUND (1000)	NOT FOUND (9)	NOT FOUND (1)	PASS
8	Above largest (max + 1)	999773243	NOT FOUND	NOT FOUND (1000)	NOT FOUND (10)	NOT FOUND (1)	PASS
9	Random present key	465969866	FOUND	FOUND (713)	FOUND (9)	FOUND (1)	PASS

#	Test case	Key	Expected	Linear	Binary	Hash	Result
10	Random absent key	906846569	NOT FOUND	NOT FOUND (1000)	NOT FOUND (10)	NOT FOUND (0)	PASS

For my registration number all three algorithms returned position 518, holding the record "Jason Joshva M J | EEE | CGPA 9.11".

Edge cases. An empty database ($n = 0$) returned NOT FOUND for both of my keys with 0 comparisons and no error. A one-record database ($n = 1$) found my registration number and correctly rejected reg no + 1 and the key just below it. All 5 edge checks passed.

Table 9.4: Random Cross-Check Against an Independent Answer Key (std::sort + std::binary_search)

n	Queries	Linear correct	Binary correct	Hash correct	My reg no found	Reg no + 1 absent
1,000	1,000	1000/1000	1000/1000	1000/1000	yes	yes
10,000	1,000	1000/1000	1000/1000	1000/1000	yes	yes
100,000	1,000	1000/1000	1000/1000	1000/1000	yes	yes
1,000,000	1,000	1000/1000	1000/1000	1000/1000	yes	yes

The answer key is built with the C++ standard library's own sort and binary search, which share no code with my algorithms. **Summary: 19 passed, 0 failed — ALL TESTS PASSED.**

9.5 Benchmarking Procedure

`./daa bench <n>` runs one dataset size and prints, in order: [1] the dataset, [2] each algorithm's preprocessing cost (time and basic operations), [3] comparisons and time per search for present and absent keys next to their theoretical values, [4] the best, worst and average case inputs, and [5] the total cost of preprocessing plus 1,000 searches, with the break-even point against linear search. Each run also writes `results/raw/bench_<n>.csv`, merged into `results/results.csv` and `results/cases.csv`, from which `src/plot_results.py` draws the graphs.

9.6 How to Reproduce

1. Set REG_NO and NAME in `src/config.h`.
2. Build from the repository root with `make` (or the single `g++` command in the README).
3. Run `./daa test`. It prints my registration number, name and the date and time first, then the three test tables and the summary line.

Figure 1: Correctness test output for linear, binary and hash search (n = 0, 1 and 1,000, cross-checked up to 1,000,000)

```

User@LAPTOP-RBLA15HT UCRT64 /c/Users/User/Downloads/DAA-A1-RegNo-Name
$ ./daa test
=====
DAA Assignment 1 (CO1) | Task 8 correctness tests | linear, binary (recursive), hash table
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:46:40 (system local time)
=====
[A] Fixed test cases on the n = 1,000 dataset (FOUND/NOT FOUND, key comparisons in brackets)
# Test case Key Expected Linear (comps) Binary (comps) Hash (comps) Result
1 Own reg no 192511209 FOUND FOUND (519) FOUND (10) FOUND (2) PASS
2 Own reg no + 1 192511210 NOT FOUND NOT FOUND (1000) NOT FOUND (10) NOT FOUND (0) PASS
3 First record (DB order) 762163974 FOUND FOUND (1) FOUND (8) FOUND (2) PASS
4 Last record (DB order) 663093799 FOUND FOUND (1000) FOUND (10) FOUND (1) PASS
5 Smallest reg no 100040148 FOUND FOUND (234) FOUND (9) FOUND (1) PASS
6 Largest reg no 999773242 FOUND FOUND (384) FOUND (10) FOUND (1) PASS
7 Below smallest (min - 1) 100040147 NOT FOUND NOT FOUND (1000) NOT FOUND (9) NOT FOUND (1) PASS
8 Above largest (max + 1) 999773243 NOT FOUND NOT FOUND (1000) NOT FOUND (10) NOT FOUND (1) PASS
9 Random present key 465969866 FOUND FOUND (713) FOUND (9) FOUND (1) PASS
10 Random absent key 906846569 NOT FOUND NOT FOUND (1000) NOT FOUND (10) NOT FOUND (0) PASS
Record retrieved for 192511209 -> position 518 | Jason Joshva M J | EEE | CGPA 9.11 (all three agree)

[B1] Edge case: empty database (n = 0)
# Test case Key Expected Linear (comps) Binary (comps) Hash (comps) Result
1 Own reg no 192511209 NOT FOUND NOT FOUND (0) NOT FOUND (0) NOT FOUND (0) PASS
2 Own reg no + 1 192511210 NOT FOUND NOT FOUND (0) NOT FOUND (0) NOT FOUND (0) PASS

[B2] Edge case: database with one record (n = 1)
# Test case Key Expected Linear (comps) Binary (comps) Hash (comps) Result
1 Own reg no 192511209 FOUND FOUND (1) FOUND (1) FOUND (1) PASS
2 Own reg no + 1 192511210 NOT FOUND NOT FOUND (1) NOT FOUND (1) NOT FOUND (0) PASS
3 Below the only key 192511208 NOT FOUND NOT FOUND (1) NOT FOUND (1) NOT FOUND (0) PASS
Record retrieved for 192511209 -> position 0 | Jason Joshva M J | EEE | CGPA 9.11 (all three agree)

[C] Random cross-check against an independent answer key (std::sort + std::binary_search)
n queries linear ok binary ok hash ok own found own+1 absent result
1,000 1000 1000/1000 1000/1000 1000/1000 yes yes PASS
10,000 1000 1000/1000 1000/1000 1000/1000 yes yes PASS
100,000 1000 1000/1000 1000/1000 1000/1000 yes yes PASS
1,000,000 1000 1000/1000 1000/1000 1000/1000 yes yes PASS
=====
SUMMARY: 19 passed, 0 failed -> ALL TESTS PASSED

```

10. TASK 9 — RECORD EXPERIMENTAL PERFORMANCE

10.1 Introduction

At 1,000,000 records a hash lookup took 0.0044 μ s, a binary search 0.163 μ s and a linear search 1,536 μ s, while the comparison counts stayed within about 7% of the Task 3 formulas at every size. Every value in this section comes from my own runs of `./daa bench` on 10 October 2026 (Figures 2–5), recorded in `results/results.csv`.

10.2 Measures Recorded

- **Input size n:** 1,000, 10,000, 100,000 and 1,000,000 records.
- **Key comparisons per search** (the basic operation), averaged separately over 500 present and 500 absent keys, plus the largest count seen.
- **Preprocessing:** time and basic operations — merge-sort comparisons for binary search, insertions for the hash table.
- **Execution time per search** in microseconds (μ s), averaged over repeated passes of the 1,000 queries lasting at least 200 ms.
- **Total cost** = preprocessing + 1,000 searches, the fair comparison when preprocessing is charged to the algorithm.

10.3 Search Performance

Table 10.1: Comparisons and Time per Search

n	Algorithm	Comparisons (key found)	Comparisons (key absent)	Max comparisons seen	Time per search (μ s)
1,000	Linear	491.62	1,000.00	1,000	0.2356
1,000	Binary	8.99	9.98	10	0.0305
1,000	Hash	1.41	0.76	6	0.0029
10,000	Linear	5,037.07	10,000.00	10,000	2.8727
10,000	Binary	12.29	13.39	14	0.0557
10,000	Hash	1.36	0.80	5	0.0027
100,000	Linear	51,139.91	100,000.00	100,000	50.0525
100,000	Binary	15.64	16.67	17	0.0775
100,000	Hash	1.38	0.79	6	0.0036
1,000,000	Linear	507,564.13	1,000,000.00	1,000,000	1,536.3384
1,000,000	Binary	18.99	19.94	20	0.1629
1,000,000	Hash	1.31	0.73	5	0.0044

All 12,000 searches across the four sizes returned the correct answer (1000/1000 at every size and for every algorithm). Hash table parameters: $m = 1,361, 13,337, 133,337$ and $1,333,357$ buckets; load factor $\alpha = 0.735$ at $n = 1,000$ and 0.750 at the other sizes; longest chain 6, 6, 8 and 7.

Reading the table. Linear search's comparisons grow tenfold with every tenfold growth in n , and an absent key always costs exactly n . Binary search adds about 3.3 comparisons per tenfold step and never exceeds $\lceil \log_2 n \rceil + 1$. The hash table stays at about 1.3–1.4 comparisons for a present key and 0.7–0.8 for an absent one at every size.

10.4 Preprocessing and Total Cost

Table 10.2: Preprocessing Cost and Total Cost of 1,000 Searches

n	Algorithm	Preprocessing (ms)	Preprocessing basic ops	Searching (ms)	Total (ms)
1,000	Linear	0.000	0	0.236	0.236
1,000	Binary	0.025	8,696 comparisons	0.031	0.056
1,000	Hash	0.003	1,000 insertions	0.003	0.006
10,000	Linear	0.000	0	2.873	2.873
10,000	Binary	0.982	120,412 comparisons	0.056	1.038
10,000	Hash	0.093	10,000 insertions	0.003	0.096
100,000	Linear	0.000	0	50.052	50.052
100,000	Binary	9.926	1,536,478 comparisons	0.077	10.003
100,000	Hash	1.176	100,000 insertions	0.004	1.180
1,000,000	Linear	0.000	0	1,536.338	1,536.338
1,000,000	Binary	128.152	18,674,676 comparisons	0.163	128.315
1,000,000	Hash	13.872	1,000,000 insertions	0.004	13.876

Even with preprocessing charged in full, the hash table had the lowest total cost at every size. At 1,000,000 records its build was 9.2 times cheaper than the merge sort (13.872 ms against 128.152 ms).

Table 10.3: Number of Searches After Which Preprocessing Pays for Itself (vs linear search)

n	Sorted index (binary search)	Hash table
1,000	125 searches	16 searches
10,000	349 searches	33 searches
100,000	199 searches	24 searches
1,000,000	84 searches	10 searches

The program divides each build time by the time saved per search compared with linear search. The hash table's break-even stays between 10 and 33 searches at every size, as Task 3 predicted (a constant), while the sorted index needs 84–349.

10.5 Terminal Output for Each Dataset Size

Figure 2: Benchmark output for n = 1,000

```

=====
DAA Assignment 1 (C01) | Task 9 benchmark | n = 1,000 records
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:48:10 (system local time)
=====
[1] Dataset : 1,000 unique records, generated in 0.3 ms, stored in random order
    Queries : 500 present + 500 absent keys - the SAME 1000 queries for all three algorithms

[2] Preprocessing (done once, before any search)
    Algorithm      work      Basic ops      Time (ms)
    Linear (iterative)  none - searches the raw records      0      0.000
    Binary (recursive)  merge sort; theory n*log2(n) = 9,965      8,696 cmp      0.025
    Hash table      m = 1,361, alpha = 0.735, max chain = 6      1,000 ins      0.003

[3] Searching: 1000 queries per algorithm (comparisons = key comparisons per search;
    us/search = average over repeated passes of the same queries, at least 200 ms)
    Algorithm      found: meas      theory      absent: meas      theory      max seen      us/search      correct
    Linear (iterative)      491.62      500.50      1000.00      1000.00      1000      0.2356      1000/1000
    Binary (recursive)      8.99      8.97      9.98      9.97      10      0.0305      1000/1000
    Hash table      1.41      1.37      0.76      0.73      6      0.0029      1000/1000
    theory: linear (n+1)/2 and n | binary log2(n+1)-1 and log2(n+1) | hash 1+alpha/2 and alpha

[4] Case analysis (Task 4): the input that triggers each case, and its comparison count
    Algo      Case      Search key      Comparisons      Theory
    Linear      Best: key is 1st record      762163974      1.00      1
    Linear      Worst: key is last record      663093799      1000.00      n = 1,000
    Linear      Worst: absent (reg no + 1)      192511210      1000.00      n = 1,000
    Linear      Own reg no (found)      192511209      519.00      its position = 519
    Linear      Average: 500 random present      -      491.62      (n+1)/2 = 500.5
    Binary      Best: key at middle of index      551004358      1.00      1
    Binary      Worst: key > largest key      999773243      10.00      floor(log2 n)+1 = 10
    Binary      Absent: reg no + 1      192511210      10.00      ~log2(n+1) = 9.97
    Binary      Own reg no (found)      192511209      10.00      <= 10
    Binary      Average: 500 random present      -      8.99      ~log2(n+1)-1 = 8.97
    Hash      Best: key at head of a chain      115640087      1.00      1
    Hash      Best absent: empty bucket      100000837      0.00      0
    Hash      Worst: end of longest chain      614992672      6.00      longest chain = 6
    Hash      Absent: reg no + 1      192511210      0.00      its chain length = 0
    Hash      Own reg no (found)      192511209      2.00      its place in its chain
    Hash      Average: 500 random present      -      1.41      1+alpha/2 = 1.367

[5] Total cost = preprocessing + 1000 searches
    Algorithm      Preprocess (ms)      Searching (ms)      Total (ms)
    Linear (iterative)      0.000      0.236      0.236
    Binary (recursive)      0.025      0.031      0.056
    Hash table      0.003      0.003      0.006
    Break-even vs linear: sorted index pays off after 125 searches, hash table after 16 searches

```

Figure 3: Benchmark output for n = 10,000

```

DAA Assignment 1 (C01) | Task 9 benchmark | n = 10,000 records
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:48:44 (system local time)
=====
[1] Dataset : 10,000 unique records, generated in 2.0 ms, stored in random order
    Queries : 500 present + 500 absent keys - the SAME 1000 queries for all three algorithms

[2] Preprocessing (done once, before any search)
    Algorithm      Work                      Basic ops      Time (ms)
    Linear (iterative)  none - searches the raw records          0          0.000
    Binary (recursive)  merge sort; theory  $n \cdot \log_2(n) = 132,877$  120,412 cmp    0.982
    Hash table         m = 13,337, alpha = 0.750, max chain = 6    10,000 ins     0.093

[3] Searching: 1000 queries per algorithm (comparisons = key comparisons per search;
    us/search = average over repeated passes of the same queries, at least 200 ms)
    Algorithm      found: meas  theory  absent: meas  theory  max seen  us/search  correct
    Linear (iterative)  5037.07    5000.50    10000.00    10000.00  10000     2.8727    1000/1000
    Binary (recursive)  12.29      12.29      13.39       13.29     14         0.0557    1000/1000
    Hash table         1.36       1.37       0.80        0.75      5          0.0027    1000/1000
    theory: linear  $(n+1)/2$  and  $n$  | binary  $\log_2(n+1)-1$  and  $\log_2(n+1)$  | hash  $1+\alpha/2$  and  $\alpha$ 

[4] Case analysis (Task 4): the input that triggers each case, and its comparison count
    Algo  Case                      Search key  Comparisons  Theory
    Linear Best: key is 1st record      639968901    1.00         1
    Linear Worst: key is last record    733612997   10000.00      $n = 10,000$ 
    Linear Worst: absent (reg no + 1)   192511210   10000.00      $n = 10,000$ 
    Linear Own reg no (found)           192511209   1149.00      its position = 1,149
    Linear Average: 500 random present -    5037.07       $(n+1)/2 = 5000.5$ 
    Binary Best: key at middle of index  549481790    1.00         1
    Binary Worst: key > largest key     999849450   14.00         $\text{floor}(\log_2 n)+1 = 14$ 
    Binary Absent: reg no + 1           192511210   13.00         $\sim \log_2(n+1) = 13.29$ 
    Binary Own reg no (found)           192511209   12.00         $\leq 14$ 
    Binary Average: 500 random present -    12.29       $\sim \log_2(n+1)-1 = 12.29$ 
    Hash Best: key at head of a chain    694964399    1.00         1
    Hash Best absent: empty bucket       100000826    0.00         0
    Hash Worst: end of longest chain     916101468    6.00        longest chain = 6
    Hash Absent: reg no + 1             192511210    0.00        its chain length = 0
    Hash Own reg no (found)             192511209    2.00        its place in its chain
    Hash Average: 500 random present     -           1.36         $1+\alpha/2 = 1.375$ 

[5] Total cost = preprocessing + 1000 searches
    Algorithm      Preprocess (ms)  Searching (ms)  Total (ms)
    Linear (iterative)  0.000          2.873          2.873
    Binary (recursive)  0.982          0.056          1.038
    Hash table         0.093          0.003          0.096
    Break-even vs linear: sorted index pays off after 349 searches, hash table after 33 searches

[6] Saved results/raw/bench_10000.csv -> merged into results/results.csv (and results/cases.csv)

```

Figure 4: Benchmark output for n = 100,000

```

=====
DAA Assignment 1 (CO1) | Task 9 benchmark | n = 100,000 records
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:49:30 (system local time)
=====

[1] Dataset : 100,000 unique records, generated in 20.2 ms, stored in random order
    Queries : 500 present + 500 absent keys - the SAME 1000 queries for all three algorithms

[2] Preprocessing (done once, before any search)
    Algorithm      Work                                     Basic ops      Time (ms)
    Linear (iterative) none - searches the raw records          0             0.000
    Binary (recursive) merge sort; theory  $n \log_2(n) = 1,660,964$  1,536,478 cmp   9.926
    Hash table      m = 133,337, alpha = 0.750, max chain = 8          100,000 ins     1.176

[3] Searching: 1000 queries per algorithm (comparisons = key comparisons per search;
    us/search = average over repeated passes of the same queries, at least 200 ms)
    Algorithm      found: meas      theory      absent: meas      theory      max seen      us/search      correct
    Linear (iterative) 51139.91 50000.50 100000.00 100000.00 100000 50.0525 1000/1000
    Binary (recursive) 15.64 15.61 16.67 16.61 17 0.0775 1000/1000
    Hash table 1.38 1.37 0.79 0.75 6 0.0036 1000/1000
    theory: linear  $(n+1)/2$  and  $n$  | binary  $\log_2(n+1)-1$  and  $\log_2(n+1)$  | hash  $1+\alpha/2$  and  $\alpha$ 

[4] Case analysis (Task 4): the input that triggers each case, and its comparison count
    Algo      Case      Search key      Comparisons      Theory
    Linear      Best: key is 1st record      889941292      1.00      1
    Linear      Worst: key is last record      890994331      100000.00       $n = 100,000$ 
    Linear      Worst: absent (reg no + 1)      192511210      100000.00       $n = 100,000$ 
    Linear      Own reg no (found)      192511209      57487.00      its position = 57,487
    Linear      Average: 500 random present      -      51139.91       $(n+1)/2 = 50000.5$ 
    Binary      Best: key at middle of index      550647813      1.00      1
    Binary      Worst: key > largest key      999989536      17.00       $\text{floor}(\log_2 n)+1 = 17$ 
    Binary      Absent: reg no + 1      192511210      17.00       $\sim \log_2(n+1) = 16.61$ 
    Binary      Own reg no (found)      192511209      16.00       $\leq 17$ 
    Binary      Average: 500 random present      -      15.64       $\sim \log_2(n+1)-1 = 15.61$ 
    Hash      Best: key at head of a chain      661618195      1.00      1
    Hash      Best absent: empty bucket      100002750      0.00      0
    Hash      Worst: end of longest chain      363471279      8.00      longest chain = 8
    Hash      Absent: reg no + 1      192511210      1.00      its chain length = 1
    Hash      Own reg no (found)      192511209      1.00      its place in its chain
    Hash      Average: 500 random present      -      1.38       $1+\alpha/2 = 1.375$ 

[5] Total cost = preprocessing + 1000 searches
    Algorithm      Preprocess (ms)      Searching (ms)      Total (ms)
    Linear (iterative) 0.000      50.052      50.052
    Binary (recursive) 9.926      0.077      10.003
    Hash table 1.176      0.004      1.180
    Break-even vs linear: sorted index pays off after 199 searches, hash table after 24 searches

```

Figure 5: Benchmark output for $n = 1,000,000$

```

=====
DAA Assignment 1 (C01) | Task 9 benchmark | n = 1,000,000 records
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:49:59 (system local time)
=====
[1] Dataset : 1,000,000 unique records, generated in 432.0 ms, stored in random order
    Queries : 500 present + 500 absent keys - the SAME 1000 queries for all three algorithms

[2] Preprocessing (done once, before any search)
    Algorithm      Work                                     Basic ops      Time (ms)
    Linear (iterative) none - searches the raw records              0              0.000
    Binary (recursive) merge sort; theory  $n \cdot \log_2(n) = 19,931,568$  18,674,676 cmp 128.152
    Hash table      m = 1,333,357, alpha = 0.750, max chain = 7 1,000,000 ins 13.872

[3] Searching: 1000 queries per algorithm (comparisons = key comparisons per search;
    us/search = average over repeated passes of the same queries, at least 200 ms)
    Algorithm      found: meas  theory  absent: meas  theory  max seen  us/search  correct
    Linear (iterative) 507564.13 500000.50 1000000.00 1000000.00 1000000 1536.3384 1000/1000
    Binary (recursive) 18.99 18.93 19.94 19.93 20 0.1629 1000/1000
    Hash table 1.31 1.37 0.73 0.75 5 0.0044 1000/1000
    theory: linear  $(n+1)/2$  and  $n$  | binary  $\log_2(n+1)-1$  and  $\log_2(n+1)$  | hash  $1+\alpha/2$  and  $\alpha$ 

[4] Case analysis (Task 4): the input that triggers each case, and its comparison count
    Algo Case Search key Comparisons Theory
    Linear Best: key is 1st record 240975095 1.00 1
    Linear Worst: key is last record 892733869 1000000.00  $n = 1,000,000$ 
    Linear Worst: absent (reg no + 1) 192511210 1000000.00  $n = 1,000,000$ 
    Linear Own reg no (found) 192511209 339457.00 its position = 339,457
    Linear Average: 500 random present - 507564.13  $(n+1)/2 = 500000.5$ 
    Binary Best: key at middle of index 550171466 1.00 1
    Binary Worst: key > largest key 999999525 20.00  $\text{floor}(\log_2 n)+1 = 20$ 
    Binary Absent: reg no + 1 192511210 20.00  $\sim \log_2(n+1) = 19.93$ 
    Binary Own reg no (found) 192511209 20.00  $\leq 20$ 
    Binary Average: 500 random present - 18.99  $\sim \log_2(n+1)-1 = 18.93$ 
    Hash Best: key at head of a chain 494675449 1.00 1
    Hash Best absent: empty bucket 100001775 0.00 0
    Hash Worst: end of longest chain 158843242 7.00 longest chain = 7
    Hash Absent: reg no + 1 192511210 1.00 its chain length = 1
    Hash Own reg no (found) 192511209 2.00 its place in its chain
    Hash Average: 500 random present - 1.31  $1+\alpha/2 = 1.375$ 

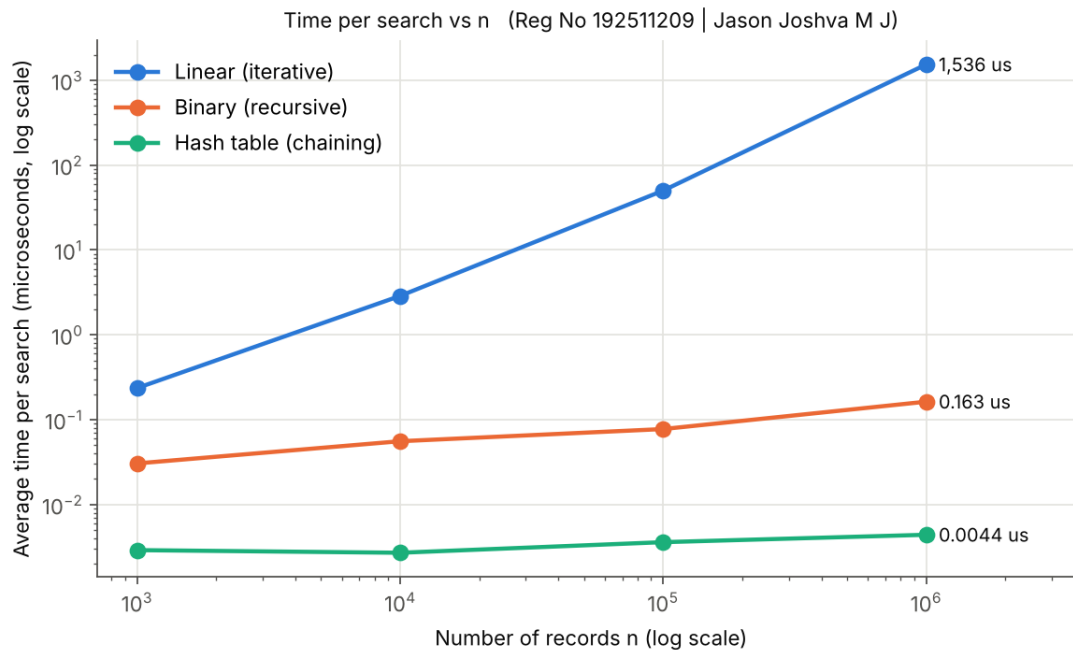
[5] Total cost = preprocessing + 1000 searches
    Algorithm Preprocess (ms) Searching (ms) Total (ms)
    Linear (iterative) 0.000 1536.338 1536.338
    Binary (recursive) 128.152 0.163 128.315
    Hash table 13.872 0.004 13.876
    Break-even vs linear: sorted index pays off after 84 searches, hash table after 10 searches

```

10.6 Graphical Analysis

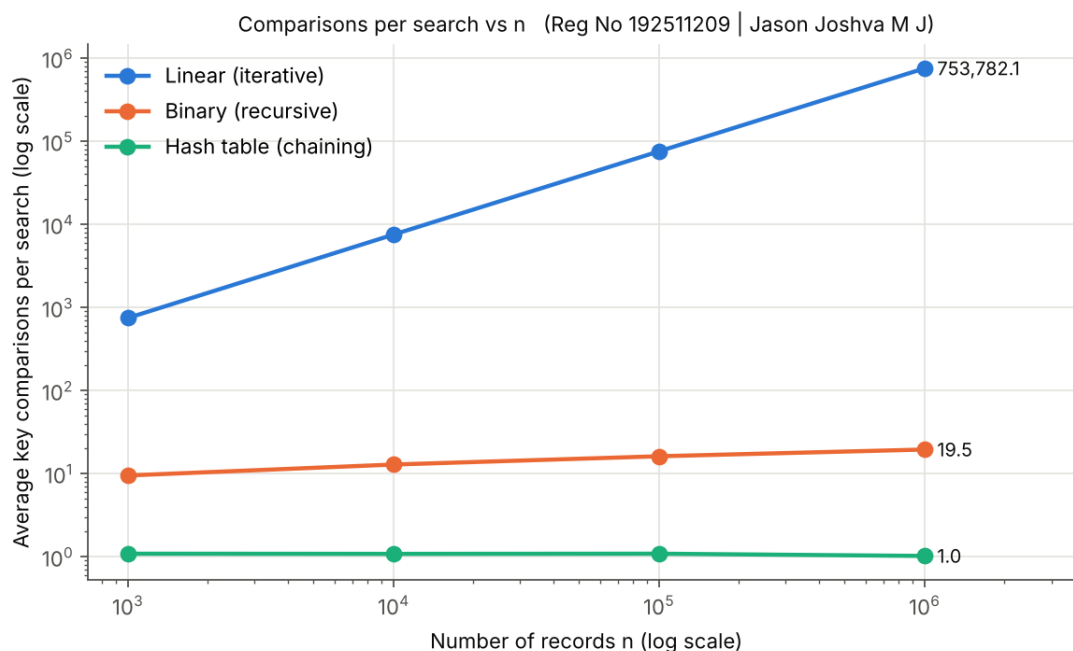
The four graphs below are drawn by `src/plot_results.py` from `results/results.csv`. All use logarithmic axes, so a straight line of slope 1 means linear growth and a flat line means constant cost.

Figure 6: Average time per search against n



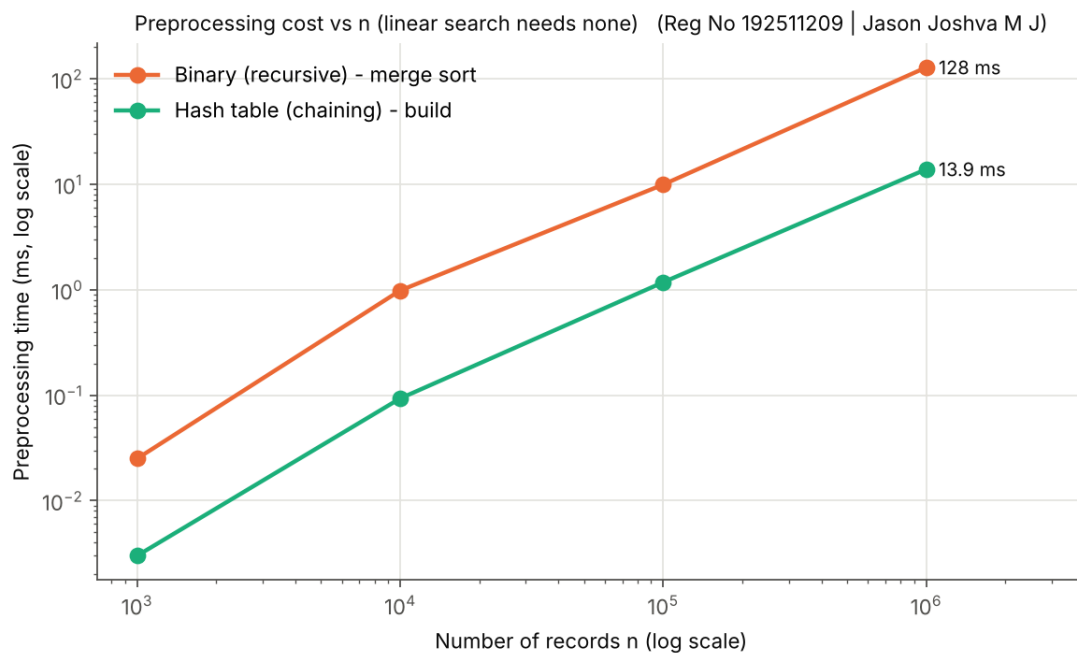
Linear search rises steeply, binary search rises gently, and the hash table is almost flat. At 10^6 records the gap between linear search and the hash table is more than five orders of magnitude.

Figure 7: Average key comparisons per search against n



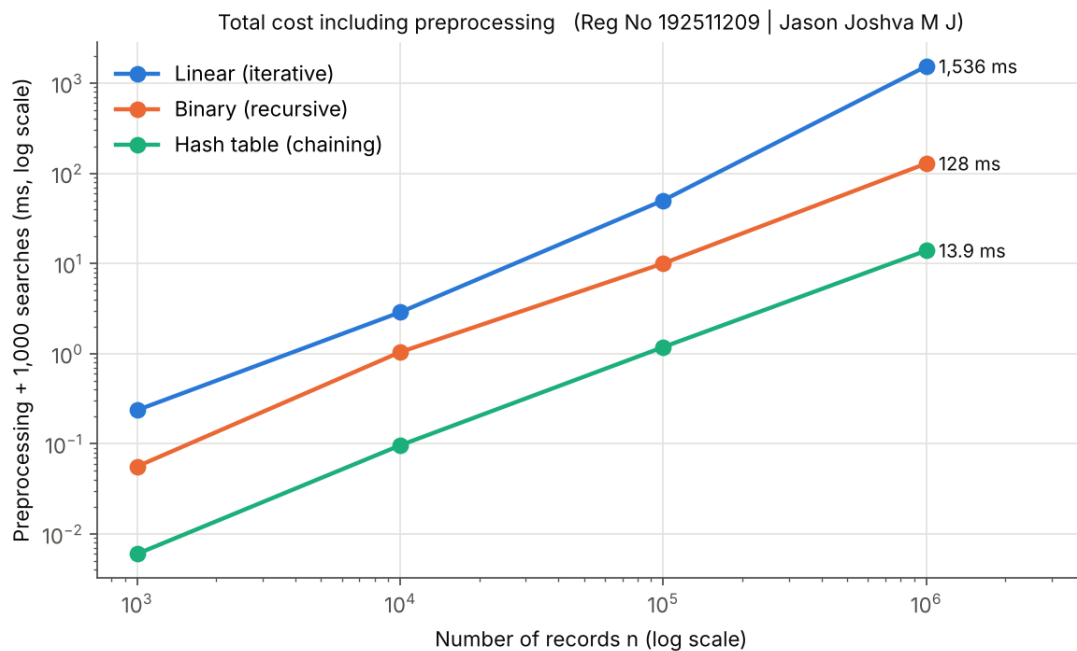
This graph shows the basic-operation counts that the times in Figure 6 are built on: a straight line for linear search, a slowly rising curve for binary search and a constant for the hash table.

Figure 8: Preprocessing cost against n (merge sort for binary search, build for the hash table)



Both preprocessing steps grow roughly in proportion to n , with merge sort about ten times more expensive at every size from 10^4 upwards. Linear search needs no preprocessing and is not plotted.

Figure 9: Total cost — preprocessing plus 1,000 searches — against n



Charging preprocessing to the algorithm does not change the order: hash table, then binary search, then linear search, at every size.

10.7 Experimental Limitations

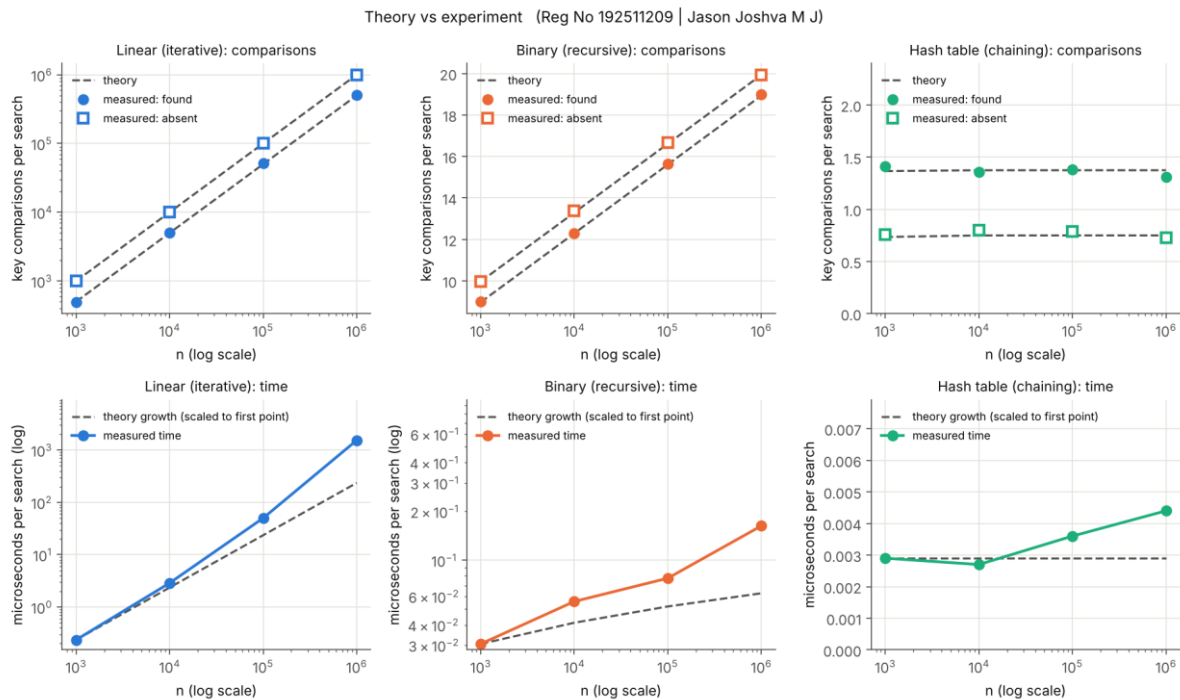
- Times are specific to my laptop, compiler and background load; the comparison counts are not, and are fully reproducible from seed 1209.
- The benchmark repeats the same 1,000 queries many times, so the data those queries touch stays in the CPU cache. Section 11 and Task 12 show that distinct keys are slower.
- Averages come from 500 random keys per case, so they vary slightly around the theoretical value.
- Very short operations (a few nanoseconds) are close to timer resolution; repeating each measurement for at least 200 ms reduces this error.

11. TASK 10 — COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

11.1 Introduction

The comparison counts agree with the theory almost exactly, so the algorithms behave as analysed. The times grow faster than the counts at large n , because the theory charges every comparison the same while real memory becomes slower as the data outgrows the CPU cache. The hash table scales best, then binary search; linear search scales worst, and becomes worse than linear at 10^6 records.

Figure 10: Top row — measured comparisons (markers) against the Task 3 formulas (dashed). Bottom row — measured time against the growth theory predicts, scaled to the $n = 1,000$ point.



11.2 Comparison Counts — Close Agreement

- Linear search:** the average comparison count grew by $10.08\times$, $10.05\times$ and $9.98\times$ for each tenfold step in n ; theory says exactly $10\times$. Present-key averages were within 2.3% of $(n+1)/2$ (507,564.13 against 500,000.5 at 10^6), and absent keys always cost exactly n .
- Binary search:** 18.99 against $\log_2(n+1) - 1 = 18.93$ for present keys and 19.94 against 19.93 for absent keys at 10^6 — within 0.8% at every size. The worst case was exactly $\lceil \log_2 n \rceil + 1$ every time: 10, 14, 17 and 20.

- **Hash table:** 1.31–1.41 against 1.375 for present keys and 0.73–0.80 against 0.75 for absent keys, within 7%, with no trend in n . The constant-cost prediction holds.
- **Merge sort:** the counted comparisons were within 0.23% of $n \log_2 n - 1.25n$ at every size (18,674,676 against 18,681,569 at 10^6).

The small differences in the averages are sampling variation. For linear search, a random key's position has standard deviation about $n/\sqrt{12}$, so the average of 500 keys varies by about $n/77$: ± 12.9 at 10^3 and $\pm 12,910$ at 10^6 . The measured deviations (-8.9 and $+7,564$) are inside one standard deviation.

11.3 Execution Time — Right Shape, Faster Growth

Table 11.1: How the Measured Time per Search Grew with Each Tenfold Step in n

Step in n	Linear: measured	Linear: theory	Binary: measured	Binary: theory (comparisons)	Hash: measured	Hash: theory
$10^3 \rightarrow 10^4$	12.2×	10×	1.83×	1.35×	0.93×	1×
$10^4 \rightarrow 10^5$	17.4×	10×	1.39×	1.26×	1.33×	1×
$10^5 \rightarrow 10^6$	30.7×	10×	2.10×	1.21×	1.22×	1×

The ordering never changed — hash table, then binary search, then linear search at every size — and the gaps widened as n grew. But every algorithm became slower per comparison as n grew: linear search from 0.32 ns to 2.04 ns per comparison, binary search from 3.2 ns to 8.4 ns per probe. Across the full 1,000-fold growth in n , the time per search grew 6,521× for linear search, 5.3× for binary search and only 1.5× for the hash table.

11.4 Why Theory and Measurement Differ

1. **Memory hierarchy.** The analysis assumes every comparison costs the same. In reality a value already in the CPU cache is read in about a nanosecond, while one in main memory takes up to a hundred. The records take 40 bytes each: 40 KB at 10^3 (fits in the fastest cache), 4 MB at 10^5 and 40 MB at 10^6 (larger than the whole cache of a typical laptop). That is why linear search's cost per comparison tripled between 10^5 and 10^6 , and its time grew 30.7× instead of 10×. Binary search and the hash table jump around memory instead of reading it in order, so at large n each probe is more likely to wait for main memory.
2. **Repeated queries stay in cache.** The Task 9 benchmark repeats the same 1,000 queries, so the index entries and hash chains they touch stay cached. With 50,000 different keys in Task 12, a hash lookup at 10^6 took 0.0382 μ s instead of 0.0044 μ s —

8.7 times slower — although its comparison count was the same (1.30). Only the memory behaviour changed.

3. **Different access patterns in preprocessing.** Merge sort reads and writes its arrays in order, which caches handle well: from 10^4 upwards its time grew $10.1\times$ and $12.9\times$ per step, close to the predicted $12.5\times$ and $12.0\times$. The hash build writes to random buckets: its time grew $12.6\times$ and $11.8\times$ against a predicted $10\times$.
4. **Fixed overheads at small n .** At $n = 1,000$ the sort took 0.025 ms and the hash build 0.003 ms, close to timer resolution, and the whole 16 KB index fits in the fastest cache. That is why their first step grew $39\times$ and $31\times$ instead of about $13\times$ and $10\times$.
5. **Fast in-cache loops.** At 10^3 linear search took only 0.32 ns per comparison — about one CPU clock cycle — because a modern processor runs several simple comparisons per cycle when the data is in cache. Theory has no such effect.
6. **Sampling and machine noise.** Averages come from 500 random keys (Section 11.2), and background processes add timing noise. Comparison counts never change between runs; times do.

11.5 Which Algorithm Scales Better

The hash table scales best: its comparison count is constant and its time stayed between 0.0027 and 0.0044 μs across a 1,000-fold growth in n . Binary search is close behind and grows slowly — doubling n adds one comparison. Linear search scales worst: its time grew $6,521\times$ for a 1,000-fold growth in n , and on real hardware it falls further behind than $\Theta(n)$ predicts once the data no longer fits in cache.

Table 11.2: Theory Against Experiment — Summary

Algorithm	Theory	Measured	Agreement
Linear search	$\Theta(n)$: $(n+1)/2$ found, n absent	Counts $\times 10$ per step; time $\times 12.2, \times 17.4, \times 30.7$	Counts exact; time grows faster than n at large n (cache)
Binary search	$\Theta(\log n)$: $\log_2(n+1) - 1$ found	18.99 vs 18.93 at 10^6 ; worst = $\lfloor \log_2 n \rfloor + 1$ exactly	Counts within 0.8%; time per probe rises with n (cache misses)
Hash table	$\Theta(1)$: $1 + \alpha/2$ found, α absent	1.31–1.41 found; 0.73–0.80 absent; time 0.0027–0.0044 μs	Counts within 7%; time nearly flat
Merge sort	$\approx n \log_2 n - 1.25n$	Within 0.23% of prediction	Exact up to sampling

12. TASK 11 — RECOMMEND AND DEFEND THE FINAL DESIGN

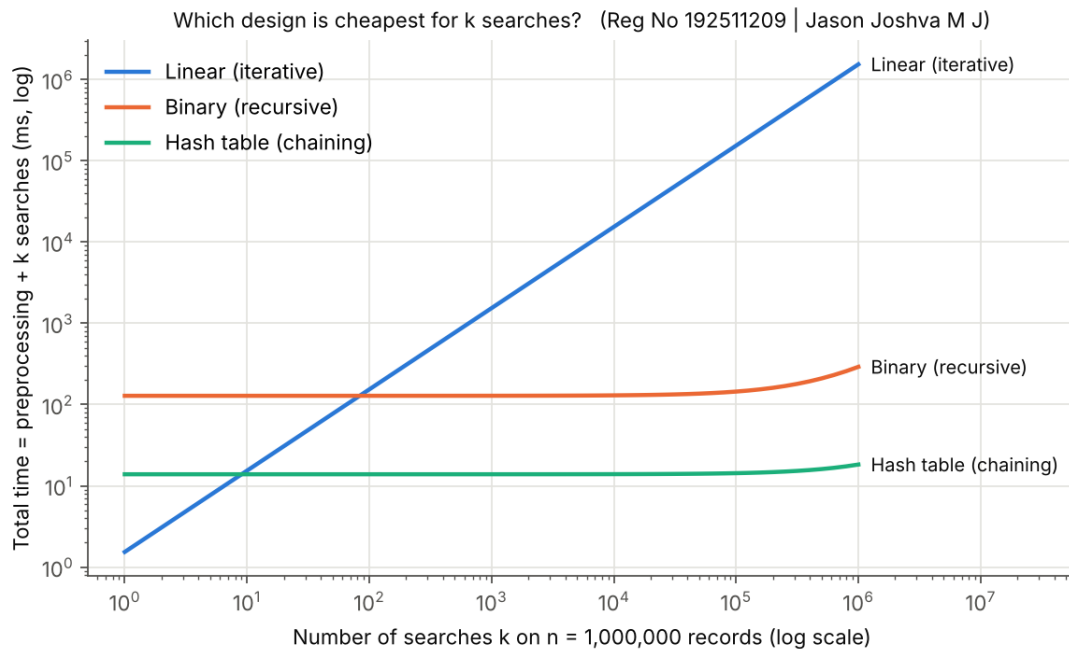
12.1 Recommendation

I recommend the **hash table with separate chaining** ($h(k) = k \bmod m$, m prime, load factor ≤ 0.75). It had the lowest search cost and the lowest total cost including preprocessing at every size tested, and its build pays for itself after 10–33 searches. Binary search over a sorted index is the fallback if the system ever needs ordered queries; linear search is only sensible for a one-off lookup on a small table.

12.2 Evidence

- **Theoretical efficiency.** Expected $\Theta(1)$ per search with $\Theta(n)$ preprocessing, against $\Theta(\log n)$ with $\Theta(n \log n)$ for binary search and $\Theta(n)$ with no preprocessing for linear search (Sections 4 and 6).
- **Measured search cost at 10^6 records.** About 1.31 key comparisons and $0.0044 \mu\text{s}$ per search, against 18.99 and $0.1629 \mu\text{s}$ for binary search and 507,564 and $1,536 \mu\text{s}$ for linear search (Table 10.1).
- **Preprocessing counted in.** Building the table took 13.872 ms, against 128.152 ms for the merge sort. Including preprocessing, 1,000 searches cost 13.876 ms in total, against 128.315 ms for binary search and 1,536.338 ms for linear search (Table 10.2).
- **Break-even.** Because the build and one linear search are both $\Theta(n)$, the table beats linear search after a near-constant number of searches — 10 to 33 at every n . The sorted index needs 84 to 349 (Table 10.3). A records office makes that many lookups within minutes.
- **Growth.** Across a 1,000-fold increase in n its comparison count stayed constant and its time grew only $1.5\times$ (Section 11).

Figure 11: Total time for k searches on 1,000,000 records, preprocessing included. Linear search is cheapest only for the first few searches; the hash table is cheapest from about 10 searches onwards.



12.3 Trade-Offs and Limitations

Table 12.1: Trade-Off Comparison

Concern	Hash table (recommended)	Binary search	Linear search
Search, expected	$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$
Search, guaranteed worst case	$\Theta(n)$ if all keys share a bucket (longest chain seen: 8)	$\Theta(\log n)$	$\Theta(n)$
Preprocessing	$\Theta(n)$: 13.872 ms at 10^6	$\Theta(n \log n)$: 128.152 ms at 10^6	None
Extra memory at 10^6 records	21.3 MB	16 MB (+16 MB while sorting)	None
Adding one record	$\Theta(1)$ amortised (occasional resize)	$\Theta(n)$ to keep the array sorted	$\Theta(1)$ append
Ordered queries (range, sorted list)	Not supported	Supported	Slow

The hash table's limitations are real but manageable for this system:

- **Weaker guaranteed worst case.** The $\Theta(n)$ worst case needs keys that all leave the same remainder modulo m . A prime m unrelated to registration-number structure makes

this practically impossible for real data. If a guaranteed response time ever mattered more than average speed, binary search's $\Theta(\log n)$ worst case would be the safer choice.

- **More memory.** About 21 bytes per record (4 per bucket plus 16 per entry), about 33% more than the sorted index, but with no temporary peak during construction. At 10 million records that is about 213 MB — acceptable for a university server.
- **No ordering.** It answers "does this registration number exist, and what is its record?" — the stated requirement — but cannot list a range of registration numbers. If that is needed later, a sorted index can be kept alongside it.

Table 12.2: Which Design Suits Which Workload

Workload	Best design	Reason (from my measurements)
A one-off lookup on a small, unsorted table	Linear search	No setup; cheaper than any build for fewer than about 10 searches
Many lookups plus sorted listings or range queries	Sorted index + binary search	$\Theta(\log n)$ worst case and ordered access; build pays off after 84–349 searches
Many lookups, infrequent updates (this system)	Hash table	≈ 1.3 comparisons per search at any n ; build pays off after 10–33 searches

12.4 Does It Remain Practical as the Database Grows?

Yes. The search cost depends on α , not n , so as long as m is enlarged with n — my `hash_insert` doubles m to the next prime when α would exceed 1 — a search stays at about one comparison whether the university has 10^3 or 10^7 students. Build time and memory grow linearly, and both are paid once rather than on every search. This recommendation does not rest on Big-O alone: the hash table also wins on measured total cost, on break-even point and on update cost, and its only weaknesses are an unlikely worst case and ordered queries that the system does not ask for.

13. TASK 12 — DAY-4 DESIGN CHALLENGE

Decision: RETAIN the hash table, MODIFY how it is operated. The algorithm stays. It becomes a persistent in-memory index that is built once and updated each day by inserting only the new records — $\Theta(k)$ work instead of a $\Theta(n)$ rebuild — with the table resized when the load factor would exceed 1.0. In my benchmark it served one hour's load of 50,000 searches in 1.91 ms of CPU time and absorbed a day's 10,000 new records in 0.458 ms.

13.1 A. Impact Analysis

Table 13.1: The Changed Operating Conditions

Requirement	Value
Database size	1,000,000 student records
Search load	50,000 searches per hour = 13.9 per second = 1,200,000 per day
Update frequency	One batch of new records per day
Main concern	Cost per search, multiplied by over a million searches a day

What has changed. The workload is now known: 50,000 searches per hour on 1,000,000 records. The update pattern is also known: new records arrive once a day as one batch, so the data is read-only for 24 hours at a time.

Which original assumptions are affected.

- *Preprocessing shared by an unknown, small number of searches.* Tasks 9–11 asked how many searches it takes for preprocessing to pay off. Now one build is shared by 1.2 million searches a day, so the break-even point is no longer the question.
- *A structure built for one session.* The original analysis built the index and then searched. Now the index must stay in memory across all searches and survive a daily update — a cost the original design never considered.
- *Warm-cache timing.* Task 9 repeated the same 1,000 queries. Real traffic means 50,000 different keys an hour, so per-search times must be measured with distinct keys.

Why search frequency now matters. The daily cost is $P + k \cdot S$ with $k = 1,200,000$. The search term is multiplied by over a million while preprocessing happens once, so the per-search cost S decides almost everything. A $\Theta(n)$ search becomes $1.2 \text{ million} \times \text{about } 540,000 \approx 6.5 \times 10^{11}$ comparisons a day.

Why rare updates matter. Preprocessing is paid once a day, so a structure that is expensive to build is acceptable. Updates arrive in one batch, so even a structure that is slow to change

one record at a time (a sorted array) could be used. And there is no need for a dynamic structure such as a balanced BST or B-tree, whose extra complexity buys cheap single inserts that this system does not need.

13.2 B. Evaluation of the Original Design Under the New Load

`./daa task12` builds each structure on 1,000,000 records and runs one hour's load of 50,000 distinct keys (45,000 present and 5,000 absent), then adds a day's batch of 10,000 new records (Figure 12).

- **Behaviour under 50,000 searches per hour.** The hash table answered all 50,000 correctly in 1.91 ms of CPU time — 0.0001% of one core's hour — averaging 1.30 comparisons and 0.0382 μ s per search, with a maximum of 7 comparisons. At this speed one core could serve about 26 million searches per second, against a demand of 13.9.
- **Preprocessing is now negligible.** The build took 16.16 ms once a day: 0.0135 μ s per search when spread over the day's 1.2 million searches — about a third of one lookup.
- **Could another approach do better?** Binary search was 7.5 times slower per search (14.30 ms for the hour, 19.05 comparisons), took 8.7 times longer to build (139.93 ms) and 10.9 times longer to absorb the daily batch (4.984 ms). Its advantages — a guaranteed $\Theta(\log n)$ worst case and ordered queries — are not needed here. Linear search is still feasible in raw CPU terms — about 55.6 s per hour, 1.54% of one core — but each lookup takes about 1.1 ms, and its cost rises with every new student and every cache miss; ten times more students would make it the bottleneck.
- **Other options considered.** A direct-address array indexed by registration number would need 900 million slots for 9-digit numbers — gigabytes of mostly empty memory. Perfect hashing gives a guaranteed $\Theta(1)$ for a fixed set, but the gain is small when the average is already about 1.3 comparisons, and it must be rebuilt for every update.

So the original algorithm remains the right one; what must change is how it is built and maintained.

13.3 C. Adaptation — Retain the Algorithm, Modify Its Operation

1. **Persistent index.** Build the table once when the system starts, keep it in memory, and let every search use it. Before, the design was "build, then search"; now the build is a once-a-day event at most.

2. **Incremental daily update.** Append the day's k new records to the database and insert only their keys into the table, instead of rebuilding it from all n records. Reserve array capacity before the batch so the arrays are not copied part-way through.
3. **Load-factor policy.** When an insert would push α above 1.0, rehash into the next prime $\geq 2m$. With 10,000 new students a day, α rises from 0.75 to 1.0 in about 33 days, so a rehash (about the cost of one rebuild, 12.234 ms) happens roughly once a month — amortised $\Theta(1)$ per insert — and a successful search never costs more than $1 + 1/2 = 1.5$ comparisons on average.

```

DAILY-UPDATE(table, DB, new[1..k])
1  RESERVE(table, table.size + k)           > no copying in the middle of the batch
2  for each record r in new do
3      append r to DB
4      HASH-INSERT(table, r.regNo, position of r in DB)

HASH-INSERT(table, key, pos)
1  if table.size + 1 > 1.0 * m then
2      REHASH(table, NEXT-PRIME(2m + 1))    > rare: about once a month
3  e <- table.size; keys[e] <- key; pos[e] <- pos
4  b <- key mod m
5  next[e] <- head[b]; head[b] <- e

```

Table 13.2: Effect of the Modification on Complexity

Cost	Original (rebuild each day)	Modified
One search	$\Theta(1)$ expected	$\Theta(1)$ expected (unchanged)
Daily update with k new records	$\Theta(n + k)$ rebuild	$\Theta(k)$ amortised
Whole day	$\Theta(n) + 1.2 \text{ million} \times \Theta(1)$	$\Theta(k) + 1.2 \text{ million} \times \Theta(1)$
Memory	$\Theta(n + m)$	$\Theta(n + m)$ (unchanged)

With $n = 10^6$ and $k = 10^4$ the daily update does about 100 times less work in theory. Measured, the incremental update took 0.458 ms against 12.234 ms for a full rebuild — 26.7 times faster.

13.4 D. Evidence-Based Defence

Table 13.3: Task 12 Benchmark on 1,000,000 Records (from Figure 12)

Measure	Hash table (chosen)	Binary search	Linear search
Build, once per day	16.16 ms	139.93 ms	none
Time for 50,000 searches (one hour of load)	1.91 ms	14.30 ms	55,607.85 ms (2,000 measured, scaled $\times 25$)
Time per search	0.0382 μ s	0.2860 μ s	1,112.157 μ s
Average comparisons per search	1.30	19.05	543,047.07
Most comparisons for one search	7	20	1,000,000
Share of one CPU core	0.0001%	0.0004%	1.5447%

Measure	Hash table (chosen)	Binary search	Linear search
CPU per day, build included	62.1 ms	483.1 ms	1,334,588.3 ms (≈ 22 min)
Build cost per search	0.0135 μ s	0.1166 μ s	0
Adding 10,000 records, incremental	0.458 ms (insert)	4.984 ms (sort batch + merge, 1,130,510 comparisons)	0.137 ms (append, no index)
Adding 10,000 records, full rebuild	12.234 ms	131.209 ms	—
Extra memory	21.3 MB	16.0 MB (+16.0 MB while sorting)	none (records: 40.0 MB)
Correct answers	50,000 / 50,000	50,000 / 50,000	2,000 / 2,000

After the daily update, 2,000 out of 2,000 check lookups were correct: every new record was found and every old answer was unchanged. After the insertions the load factor was 0.757.

Why the decision follows from the evidence:

- **Mathematics.** Each day costs $P + 1.2 \text{ million} \times S$. With $S = \Theta(1)$ for the hash table against $\Theta(\log n)$ for binary search, the hash table makes 14.7 times fewer comparisons per search (1.30 against 19.05), and the gap widens as n grows.
- **Experiment.** On 50,000 distinct keys it was 7.5 times faster than binary search and about 29,000 times faster than linear search (the program's own summary line). The gap to binary search is smaller than the comparison ratio because at this size each lookup is limited by memory access rather than by comparisons (Section 11.4).
- **Search frequency.** At 13.9 searches per second both indexed designs are idle more than 99.99% of the time. What separates them is latency, headroom and how the cost grows — and the hash table leads on all three.
- **Preprocessing.** Spread over 1.2 million searches, building the table costs 0.0135 μ s per search. With incremental updates the daily cost falls to 0.458 ms, so preprocessing no longer affects the choice.
- **Dataset size.** The search cost does not depend on n , the build and memory grow linearly, and the resize policy keeps α bounded as the university grows.
- **Trade-offs accepted.** About 21 bytes of memory per record, no range queries, and a worst case excluded only in practice (the longest chain reached in 50,000 lookups was 7). If ordered listings are needed later, a sorted index can be maintained alongside it with one 5 ms merge per day.

Figure 12: Task 12 benchmark output — 50,000 searches on 1,000,000 records and the daily update of 10,000 records

```

$ ./daa task12
=====
DAA Assignment 1 (C01) | Task 12 | 50,000 searches/hour on 1,000,000 records, new records once a day
Reg No: 192511209 | Name: Jason Joshva M J | Dataset seed: 1209 (last 4 digits of reg no)
Run at: 2026-10-10 19:50:43 (system local time)
=====
[1] Workload: 1,000,000 records (generated in 459 ms). 50,000 searches per hour = 13.9 per second.
    Query mix: 45,000 present + 5,000 absent (most lookups are for students who exist).
    Per day: 1,200,000 searches, but only ONE batch of new records -> index built once per day.

[2] Preprocessing, done once per day
    Binary search : merge sort of the index      139.93 ms (18,674,676 comparisons)
    Hash table    : build 1,333,357 buckets      16.16 ms (alpha = 0.750, longest chain = 7)
    Extra memory  : sorted index 16.0 MB (+16.0 MB temporary while sorting), hash table 21.3 MB, records 40.0 MB

[3] Running the hourly load: 50,000 searches
    Algorithm      searches run      time for 50,000      us/search      avg comps      max comps      correct
    Hash table     50,000             1.91 ms           0.0382          1.30            7             50,000/50,000
    Binary (recursive) 50,000             14.30 ms          0.2860          19.05           20            50,000/50,000
    Linear (iterative) 2,000             55607.85 ms       1112.1570       543047.07       1000000       2,000/2,000 <- measured on 2,000, scaled x25

[4] Cost of the new operating conditions
    Algorithm      build/day (ms)      CPU/hour (ms)      % of 1 hour      CPU/day (ms)      build per search(us)
    Hash table     16.16              1.91              0.0001%          62.1              0.0135
    Binary (recursive) 139.93            14.30             0.0004%          483.1             0.1166
    Linear (iterative) 0.00              55607.85          1.5447%          1334588.3         0.0000

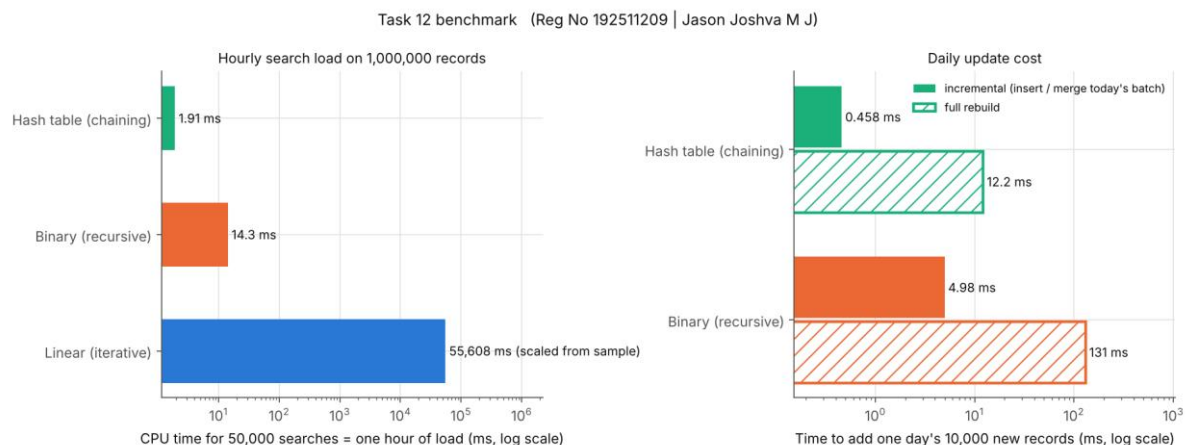
[5] Daily update: 10,000 new records added (database grows to 1,010,000)
    Algorithm      Incremental update      time (ms)      Full rebuild      time (ms)
    Linear (iterative) append records, no index      0.137 -
    Binary (recursive) sort batch + merge (1,130,510 comps)      4.984 merge sort all records      131.209
    Hash table       insert 10,000 keys; alpha now 0.757      0.458 rebuild whole table      12.234
    Check after update: 2000/2000 lookups correct (new keys found, old answers unchanged)

[6] Summary (from the numbers above)
    Hash table is 7.5x faster per search than binary search and 29081x faster than linear search.
    Building the hash table once a day costs 0.0135 us per search when spread over 1,200,000 searches.
    Linear search would need 55.6 s of CPU every hour; the hash table needs 1.91 ms.

[7] Saved results/task12.csv

```

Figure 13: CPU time for one hour of searches, and the cost of the daily update (incremental against full rebuild), both on log scales



13.5 E. Final Decision

RETAIN the hash table and MODIFY its operation: keep one persistent table in memory, insert each day's new records incrementally, and rehash only when the load factor would exceed 1.0. This follows from the complexity analysis ($\Theta(1)$ per search for 1.2 million searches a day), from the measurements (1.91 ms per hour of load, 7.5 times faster than binary search), from the low update frequency (0.458 ms per daily batch), and from the dataset size (cost independent of n). A redesign is not needed, because no alternative improves on the retained algorithm under these conditions.

14. CONCLUSION

This project designed, analysed and tested three search strategies for a student record system. Linear search needs no preparation but makes $\Theta(n)$ comparisons per search. Recursive binary search makes $\Theta(\log n)$ comparisons — exactly $\lfloor \log_2 n \rfloor + 1$ in the worst case, as solved from its recurrence — but needs a $\Theta(n \log n)$ sort first. The hash table makes an expected $1 + \alpha/2$ comparisons, independent of n , after a $\Theta(n)$ build.

My measurements confirmed the operation counts almost exactly: within 2.3% for linear search, 0.8% for binary search and 7% for the hash table, with every best and worst case matching its prediction. The times followed the same order but grew faster than the counts at large n , because the theory treats every memory access as equally fast while real memory slows down once the data outgrows the CPU cache.

The hash table was the best design at every size, even with its preprocessing counted: at 1,000,000 records, 1,000 searches cost 13.876 ms in total against 128.315 ms for binary search and 1,536.338 ms for linear search. For the Task 12 workload of 50,000 searches per hour with one daily update, the hash table is retained and operated as a persistent index with incremental updates, serving an hour's load in 1.91 ms and a day's new records in 0.458 ms.

Table 14.1: Final Findings

Question	Finding
Simplest method, no setup	Linear search
Best guaranteed worst case	Binary search: $\lfloor \log_2 n \rfloor + 1$ comparisons
Lowest search cost	Hash table: about 1.3 comparisons and 0.0044 μ s at 10^6 records
Lowest total cost, preprocessing included	Hash table, at every size tested
Break-even against linear search	Hash table: 10–33 searches; sorted index: 84–349 searches
Recommended design (Task 11)	Hash table with separate chaining, m prime, $\alpha \leq 0.75$
Decision for Task 12	Retain the hash table; modify it to a persistent index with incremental daily updates
Main lesson	Choose by total cost for the real workload — preprocessing, search frequency, update frequency and memory — not by Big-O alone

15. APPENDIX — REPRODUCIBILITY AND SUBMISSION

15.1 Execution Procedure

The project is written in C++17. On Windows it is built and run in the **MSYS2 UCRT64** terminal; every command below is typed there, from the repository root.

Step 1 — Install the compiler and build tool (once).

```
pacman -Syu
pacman -S --needed mingw-w64-ucrt-x86_64-gcc make
```

Step 2 — Set my details in `src/config.h` (`REG_NO = 192511209`, `NAME = "Jason Joshva M J"`), then build:

```
make
```

Step 3 — Run the correctness tests (Figure 1).

```
./daa test
```

Step 4 — Run the benchmarks (Figures 2–5 and 12).

```
./daa bench 1000
./daa bench 10000
./daa bench 100000
./daa bench 1000000
./daa task12
```

Step 5 — Draw the graphs from the CSV files (Figures 6–11 and 13). This needs Python with matplotlib (`pacman -S --needed mingw-w64-ucrt-x86_64-python-matplotlib`).

```
python src/plot_results.py
```

15.2 Repository Structure

```
DAA-A1-192511209-JasonJoshvaMJ/
|-- README.md           name, reg no, how to run, algorithms chosen
|-- Makefile
|-- report/             DAA_A1_192511209.pdf
|-- src/                config.h, common.h, algo1_linear.cpp, algo2_binary.cpp,
                        algo3_hash.cpp, dataset_generator.cpp, benchmark.cpp,
                        plot_results.py
|-- data/               sample dataset (./daa gen 1000)
|-- results/            results.csv, cases.csv, task12.csv, graphs/
`-- screenshots/        task08_correctness_test.png, task09_runtime_<n>.png,
                        task12_benchmark.png
```

15.3 Final Submission Checklist

No.	Requirement	Status
1	Registration number, name and GitHub link on the first page	
2	All twelve tasks included, in order	
3	Pseudocode and mathematical derivations present	

No.	Requirement	Status
4	Best, average and worst cases explained with their inputs	
5	Recurrence formulated from the code and solved	
6	Correctness output (Figure 1) with reg no, name and date/time visible	
7	Benchmark screenshots for all four sizes, numbers matching Tables 10.1–10.3	
8	Graphs generated from my own results.csv, axes labelled	
9	Task 12 benchmark (Figure 12) and decision included	
10	Repository has README, at least 5 meaningful commits, and this PDF	
11	Same PDF uploaded to the repository and to Google Classroom	